Learning-Based Model Predictive Control for Cooperative Spacecraft Swarm Reconfiguration

HE REN
Beihang University, Beijing, China

RUI ZHONG
HAICHAO GUI
Beihang University, Beijing, China
Ministry of Education, Beijing, China

This article presents a reference-free and decentralized algorithm for spacecraft swarm reconfiguration that simultaneously ensures collision avoidance and fuel efficiency. The objective is to transfer a large number of capability-constrained spacecraft from arbitrary initial formations to a desired target configuration. To enhance autonomy and real-time responsiveness, a learning-based model predictive control (MPC) framework is developed, in which reinforcement learning (RL) is integrated into the MPC structure. By leveraging the full-horizon maneuver schedule generated by the RL agent, the MPC algorithm can compute maneuver strategies directly toward the target configuration at the terminal time, without relying on a predefined reference trajectory. In contrast to prior works that employ continuous-thrust control with a fixed discretization interval Δt, the proposed method utilizes impulsive maneuvers with an adaptively adjusted Δt. This design significantly improves the flexibility of the algorithm and reduces the online computation frequency when handling large-scale reconfiguration scenarios. To address the computational complexity of collision-avoidance constraints in large-scale swarms, a fast-constructible collision-avoidance region is developed based on reachable sets. The recursive feasibility and asymptotic stability of the overall system are guaranteed through a Lyapunov-based analysis incorporating artificial constraints. Finally, the proposed approach is validated through extensive Monte Carlo simulations and benchmark comparisons with existing state-of-the-art methods.

Received 3 June 2025; revised 2 September 2025; accepted 19 October 2025. Date of publication 23 October 2025; date of current version 14 January 2026.

DOI. No. 10.1109/TAES.2025.3624705

Refereeing of this contribution was handled by R. Dai.

This work was supported in part by the National Key R&D Program of China under Grant 2024YFB3909900, in part by the National Natural Science Foundation of China under Grant 12422214, and in part by the Taihu Lake Innovation Fund for Future Technology.

Authors' addresses: He Ren is with the School of Astronautics, Beihang University, Beijing 100191, China, E-mail: (renhe_email@163.com); Rui Zhong and Haichao Gui are with the School of Astronautics, Beihang University, Beijing 100191, China, and also with the Key Laboratory of Spacecraft Design Optimization and Dynamic Simulation Technology, Ministry of Education, Beijing 100816, China, E-mail: (zhongruia@163.com; hcgui@buaa.edu.cn). *(Corresponding authors: Rui Zhong; Haichao Gui.)*

I. INTRODUCTION

With the continuous advancement of space technology, satellites are playing an increasingly significant role in various aspects of human life. Accordingly, the design and control of a single, monolithic satellite are becoming progressively more complex, accompanied by rising launch costs and operational risks. To address these limitations, the concept of spacecraft swarms [1], [2], [3] has attracted growing interest from the research community, offering a promising alternative through distributed architectures and cooperative control.

In recent decades, the widespread applications of spacecraft swarms in complex missions, such as synthetic aperture radar [4], the silicon wafer integrated femtosatellites (SWIFT) [5], and Starling for communication and navigation [6], have stimulated significant research interest in this field. Compared with the single, monolithic satellite, the spacecraft swarms consist of a large number of small-size, low-cost spacecraft, feature shorter development cycles, stronger survivability and reliability, lower launch costs and risks, and superior mission precision and adaptability [7], [8], [9]. However, despite these advantages, each individual spacecraft within the swarm is inherently constrained in terms of its capabilities, posing substantial challenges for the design of effective guidance and control strategies [10], [11], [12], [13]. In addition, the close proximity of spacecraft within the swarm demands high real-time responsiveness and autonomy from the control system to ensure collision avoidance. The limited propulsion and onboard computational resources, combined with the complexity of the space environment, make the development of a collision-free, fuel-efficient, and computationally efficient control strategy, which is a critical requirement for the successful deployment of spacecraft swarms.

Previous studies on spacecraft formation flying [14], [15], [16], [17] have proposed various control strategies. However, these approaches typically focus on small-scale formations, often involving no more than a dozen spacecraft. Only a limited number of works have addressed the cooperative control problem in the context of large-scale spacecraft swarms. For instance, Cui et al. [10] developed a reachable-set-based trajectory planning algorithm to generate transfer solutions for spacecraft swarms, utilizing sequential convex programming (SCP) with trust-region constraints to obtain optimal trajectories. Chen et al. [18] proposed a distributed trajectory optimization framework for large-scale spacecraft formation reconfiguration, employing a pseudospectral method to solve the underlying trajectory optimization problem. Morgan et al. [19] designed a multiburn maneuver strategy to ensure collision-free orbits for hundreds of spacecraft under atmospheric

drag perturbations. While these methods are capable of producing near-optimal solutions for swarm reconfiguration, their computational complexity grows rapidly with the number of spacecraft, making it challenging for them to meet the real-time performance requirements of time-sensitive mission scenarios.

Another notable observation is that, in time-critical missions with strict duration constraints, impulsive maneuvers offer superior maneuverability and simpler control execution [20]. However, impulsive maneuver strategies are often overlooked. This preference is largely due to the inherent discreteness of impulse control, which makes multi-impulse trajectory optimization particularly challenging. Lippe and D'Amico [21] proposed a control architecture combining artificial potential functions (APFs) with convex optimization techniques. Renevey and Spencer [22] introduced an APF-based control approach for spacecraft formation establishment; however, their method does not explicitly account for fuel consumption in the optimization process. Wang et al. [23] further developed a self-organizing control strategy based on APFs in terms of relative orbital elements, which is particularly effective for tracking moving targets. A common characteristic among these impulse-based algorithms is that the impulse command Δv is generated based on APF guidance, while the sampling interval Δt remains fixed. However, a small Δt results in a substantial communication burden due to frequent APF evaluations, whereas a large Δt imposes greater demands on the spacecraft's propulsion system.

To address the challenges of relative motion in close-proximity spacecraft operations, several emerging techniques have been adopted, among which model predictive control (MPC) is a prominent example. MPC [24], [25] has been an active research area for decades. Its finite-horizon prediction and receding horizon control scheme offer excellent real-time performance and inherent obstacle avoidance capabilities, which have motivated its application to spacecraft relative motion control problems [26], [27], [28]. In spacecraft swarm control, MPC is often coupled with reference trajectory planning methods—most of which are numerical in nature—such as SCP [29], [30] and the Legendre pseudospectral method [31]. However, the time required to compute reference trajectories in complex environments remains prohibitively long, limiting the applicability of such approaches in real-time scenarios. Some studies attempt to alleviate this issue by planning only a short segment of the future trajectory [29]. Another promising technique is reinforcement learning (RL), which has been applied to various space missions, including interception, rendezvous [32], [33], [34], and satellite constellation management [35]. RL-based approaches have also been explored for spacecraft swarm reconfiguration. For example, Sun et al. [36] proposed a cooperative controller utilizing an actor neural network, where both soft and hard constraints are embedded into the RL agent. Sabol et al. [37] introduced a machine-learning-based method for coordinating multispacecraft motion via passive relative orbits. Meng et al. [38] developed a distributed RL-based optimal control strategy for obstacle avoidance in spacecraft clusters. Despite the promising results achieved by RL in various applications, its stability and reliability as an end-to-end control approach remain a key concern in engineering practice.

To further harness the advantages of both MPC and RL, the authors in [39] and [40] have proposed learning-based MPC algorithms. Compared to traditional MPC or standalone RL approaches, learning-based MPC not only improves the fidelity of system dynamics models and enhances the flexibility of controller design, but also reduces reliance on human expertise while ensuring controller safety and stability. Its core value lies in the integration of machine learning and optimization-based control, thereby forming an intelligent control framework capable of continuously improving closed-loop performance. This makes learning-based MPC particularly well-suited for complex dynamical systems and mission scenarios with stringent safety requirements. Learning-based MPC approaches are generally classified into three categories: learning the system dynamics [41], [42], [43], learning the controller design [44], [45], [46], and deriving safety guarantees for learning-enhanced controllers [47], [48], [49]. However, current research predominantly focuses on single-agent systems, with relatively limited exploration of its extension to multi-agent collaborative systems that require coordinated control policies. Furthermore, existing approaches primarily utilize machine learning techniques as auxiliary tools to optimize specific components within the MPC framework, without fully exploiting the intelligence and adaptability inherent to machine learning. These observations suggest that learning-based MPC still possesses considerable untapped potential for future research and application.

The objective of this article is to develop a learning-based MPC algorithm for multiagent systems, specifically spacecraft swarms, to enable fuel-efficient, collision-free reconfiguration while remaining suitable for deployment on capability-constrained spacecraft with limited propulsion and onboard computational resources. The proposed algorithm offers three major advantages. First, it is decentralized: each spacecraft autonomously determines its transfer plan based solely on locally observed states within a limited range. Second, the algorithm is trajectory-free: it employs a full-horizon prediction strategy with the help of RL that eliminates dependence on predefined reference trajectories, thereby enhancing real-time performance and adaptability in complex and dynamic environments. Third, the algorithm integrates impulsive maneuvering with adaptive sampling intervals, dynamically adjusting the maneuver interval Δt based on the spacecraft's current state, which can significantly reduce the frequency of optimization problem solving, conserve onboard computational resources, and extend the operational lifespan of the spacecraft. In addition, a reachable-set-based method [50] is introduced for the rapid construction of collision-avoidance regions, thereby avoiding the computational overhead of evaluating relative positions with all other spacecraft during optimization. Artificial constraints are incorporated into the MPC framework

to guarantee recursive feasibility and ensure the stability of the proposed learning-based MPC approach. The key contributions and distinguishing features of this work are summarized as follows.

1) We propose a reference-free MPC design for the reconfiguration of large-scale spacecraft swarms. To the best of the authors' knowledge, most existing MPC formulations for spacecraft formation control rely on a predefined reference trajectory [29], [30], [31], which may not be available or practical in large-scale reconfiguration missions. In contrast, the proposed algorithm generates control policies in real time based solely on locally observed states. This design enables the swarm to accomplish emergent tasks in unfamiliar environments without dependence on ground-based commands, thereby substantially improving system autonomy and intelligence.
2) Unlike previous approaches [28], [41], [45] that utilize RL primarily as an auxiliary tool to modify system dynamics or the stage cost function in MPC, our method leverages RL to construct a full-horizon prediction strategy that mitigates the limitations associated with finite control horizons. In parallel, RL introduces a flexible triggering mechanism via an adaptive time interval Δt, which reduces the frequency of optimization, thereby conserving onboard computational resources while enabling more suitable solutions in complex mission scenarios.
3) A reachable-set-based method is proposed for the rapid construction of collision-avoidance regions. In contrast to existing optimization algorithms that require sequential computation of pairwise collision-avoidance constraints among spacecraft, the proposed approach supports batch processing, thereby significantly reducing computational complexity and improving scalability for large-scale swarms.

The rest of this article is organized as follows. Section II provides the formal definition and detailed model of the spacecraft swarm optimization problem. Section III introduces the learning-based MPC algorithm and the method for calculating the reachable region, detailing the framework and the training of all networks. To evaluate the performance of our algorithm, we compare it with several baseline algorithms. Section IV presents the simulation results. Finally, Section V concludes this article.

II. PRELIMINARIES AND PROBLEM FORMULATION

Swarm reconfiguration refers to the process of transferring a large number of spacecraft from their initial orbits to a desired formation, while ensuring collision avoidance and minimizing total fuel consumption throughout the maneuver. To define the variables and constraints involved in this problem, two coordinate systems are introduced. First, the Earth-centered inertial (ECI) coordinate system, denoted as $O_{\mathcal{I}} - \hat{X}\hat{Y}\hat{Z}$, is employed to describe the position of the chief spacecraft or a virtual reference trajectory known as the chief orbit. $\hat{X}$ axis points toward the vernal equinox, the $\hat{Z}$ axis is aligned with the Earth's rotational axis and points toward the North Pole, and the $\hat{Y}$ axis completes the right-handed orthonormal basis. Second, a local-vertical local-horizontal (LVLH) coordinate frame, denoted as $O_{\mathcal{L}} - \hat{x}\hat{y}\hat{z}$, is defined relative to the chief spacecraft. In this frame, the $\hat{x}$ axis is aligned with the radial direction and points outward from the center of the Earth, the $\hat{z}$ axis is aligned with the angular momentum vector of the chief orbit, and the $\hat{y}$ axis completes the right-handed frame by pointing in the along-track direction. The relationship between the two coordinate systems is illustrated in Fig. 1.

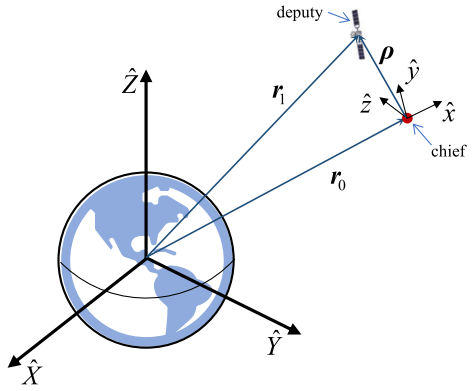

Fig. 1. ECI $O_{\mathcal{I}} - \hat{X}\hat{Y}\hat{Z}$ frame and LVLH $O_{\mathcal{L}} - \hat{x}\hat{y}\hat{z}$.

The positions of the chief spacecraft and the deputy are $\boldsymbol{r}_0$ and $\boldsymbol{r}_1$, respectively. The relative position vector of the deputy with respect to the chief is defined as $\boldsymbol{\rho} = [x, y, z]^T$

$$\boldsymbol{\rho} = \boldsymbol{r}_1 - \boldsymbol{r}_0. \tag{1}$$

The inertial equations of motion of the deputy relative to the chief are given by

$$\begin{aligned}\ddot{\boldsymbol{\rho}} &= -\frac{\mu\left(\boldsymbol{r}_0+\boldsymbol{\rho}\right)}{\|\boldsymbol{r}_0+\boldsymbol{\rho}\|^3} + \frac{\mu}{r_0^3}\boldsymbol{r}_0 \\ &= \frac{d^2\boldsymbol{\rho}}{dt^2} + 2\times\boldsymbol{\omega}\times\frac{d\boldsymbol{\rho}}{dt} + \frac{d\boldsymbol{\omega}}{dt}\times\boldsymbol{\rho} + \boldsymbol{\omega}\times\boldsymbol{\omega}\times\boldsymbol{\rho}\end{aligned} \tag{2}$$

where μ is the Earth's gravitational constant and $\boldsymbol{\omega}$ denotes the angular velocity vector of frame $\mathcal{L}$ relative to frame $\mathcal{I}$.

By substituting $\boldsymbol{\omega} = [0, 0, \dot{\theta}_0]^T$ and $\boldsymbol{r}_0 = [r_0, 0, 0]^T$ into (2), the componentwise expressions for the relative motion become

$$\begin{aligned}\ddot{x} - 2\dot{\theta}_0\dot{y} - \ddot{\theta}_0 y - \dot{\theta}^2 x &= -\frac{\mu\left(r_0+x\right)}{\left[(r_0+x)^2+y^2+z^2\right]^{\frac{3}{2}}} + \frac{\mu}{r_0^2} \\ \ddot{y} + 2\dot{\theta}_0\dot{x} + \ddot{\theta}_0 x - \dot{\theta}^2 y &= -\frac{\mu y}{\left[(r_0+x)^2+y^2+z^2\right]^{\frac{3}{2}}} \\ \ddot{z} &= -\frac{\mu z}{\left[(r_0+x)^2+y^2+z^2\right]^{\frac{3}{2}}}.\end{aligned} \tag{3}$$

ASSUMPTION 1 The chief spacecraft is assumed to follow a circular orbit, and the relative distance between the deputy and the chief is significantly smaller than their respective distances from the Earth's center.

Under this assumption, the linearized unperturbed relative dynamics can be described by the Clohessy–Wiltshire (CW) equations

$$\begin{cases} \ddot{x} - \omega\dot{y} - 3\omega^2 x = u_x \\ \ddot{y} + 2\omega\dot{x} = u_y \\ \ddot{z} + \omega^2 z = u_z \end{cases} \tag{4}$$

where $\omega = \sqrt{\mu/r_0^3}$ represents the mean orbital angular velocity of the chief. In the case where the deputy employs impulsive maneuvers and is not subject to external disturbances (i.e., $u_i = 0$, for $i = x, y, z$), let the state vector at time t_{k-1} be defined as $\boldsymbol{\zeta}(t_{k-1}) = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^T$. The analytical solution of the state at time t_k is given by

$$\boldsymbol{\zeta}(t_k) = \boldsymbol{\Phi}(\Delta t)\,\boldsymbol{\zeta}\left(t_{k-1}\right). \tag{5}$$

The state transition matrix $\boldsymbol{\Phi}(\Delta t)$ is given as (6) shown at the bottom of this page, where $\Delta t = t_k - t_{k-1}$, $\psi = \omega\Delta t$, $s = \sin(\psi)$, and $c = \cos(\psi)$. If the deputy executes an impulsive maneuver at time t_k with a velocity change $\boldsymbol{\Delta v}(t_k) = [\Delta\dot{x}, \Delta\dot{y}, \Delta\dot{z}]^T$, then the state transition can be described as

$$\boldsymbol{\zeta}(t_k) = \boldsymbol{\Phi}(\Delta t)\,\boldsymbol{\zeta}\left(t_{k-1}\right) + \mathbf{B}\boldsymbol{\Delta v}(t_k) \tag{7}$$

where the input matrix $\mathbf{B}$ is defined as

$$\mathbf{B} = \begin{bmatrix} \mathbf{0}_{3\times3} \\ \mathbf{I}_{3\times3} \end{bmatrix}. \tag{8}$$

The spacecraft swarm reconfiguration problem involves transferring all spacecraft from an initial formation to the predefined target configuration without collisions. The objective of the control policy is to minimize the cumulative fuel consumption resulting from impulsive maneuvers. Consider a swarm consisting of N spacecraft. The initial state of the ith spacecraft at time t_0 is denoted by $\boldsymbol{\zeta}_i(t_0)$, and it is required to reach the designated target state $\boldsymbol{\zeta}_i(t_f)$ at the final time t_f through a sequence of impulse maneuvers. Accordingly, the swarm reconfiguration problem can be formulated as the following optimization problem.

Problem 1

$$\min_{i=1,\ldots,N} \sum_{i=1}^{N} \sum_{k=1}^{\mathrm{K}_i} \left\|\boldsymbol{\Delta v}_i(t_k)\right\|$$
$$\text{s.t. } \boldsymbol{\zeta}_i\left(t_{k+1}\right) = \boldsymbol{\Phi}(\Delta t)\,\boldsymbol{\zeta}_i(t_k) + \mathbf{B}\boldsymbol{\Delta v}_i\left(t_{k+1}\right) \tag{9}$$

subject to

$$\left\|\boldsymbol{\Delta v}_i(t_k)\right\|_\infty \le I_{\max}$$
$$t_k = 1, 2, \ldots, \mathrm{K}_i; \quad i = 1, 2, \ldots, N \tag{10}$$
$$\left\|\boldsymbol{r}_i(t) - \boldsymbol{r}_j(t)\right\| > d_{\min} \tag{11}$$

$$\boldsymbol{\zeta}_i(t_0) = \boldsymbol{\zeta}_i(t_0)\ ;\ \boldsymbol{\zeta}_i\left(t_{\mathrm{K}_i}\right) = \boldsymbol{\zeta}_i\left(t_f\right). \tag{12}$$

Here, K_i denotes the number of impulsive maneuvers executed by the ith spacecraft and $I_{\max}$ represents the upper bound on the magnitude of each impulsive velocity change. The constraint in (11) ensures that a minimum safety distance $d_{\min}$ is maintained between each pair of spacecraft at all times.

III. LEARNING-BASED MPC CONTROL DESIGN

This section details the design of the proposed learning-based MPC framework. We begin by outlining the motivation for integrating RL with MPC, followed by a description of how a collision-free region is constructed using reachable sets. Lastly, we discuss the recursive feasibility and stability guarantees of the learning-enhanced MPC scheme.

A. Integration of RL and MPC

Traditional MPC methods rely on predefined reference trajectories to formulate the cost function. The optimal control sequence is obtained by minimizing this cost function over a finite prediction horizon, and only the first control input is applied to the system at each time step. However, this approach suffers from two critical limitations. First, generating suitable reference trajectories is often nontrivial, particularly in high-dimensional, constrained environments, such as spacecraft swarms, where safety and performance must be simultaneously addressed. Second, due to the limited prediction horizon inherent to MPC, the resulting control actions may only yield locally optimal solutions, failing to account for longer term objectives and global mission performance.

Therefore, we propose an MPC framework that eliminates the dependence on reference trajectories and extends its predictive capability over the full remaining time horizon. Specifically, at any given time t ($t \in [t_0, t_f]$), we assume that the deputy i is allowed to execute a total of $\mathscr{K}+1$ impulsive maneuvers within the remaining time window $t_f - t$. The corresponding maneuver time sequence (MTS) given by the RL agent is defined as $T = [t_0, t_1, t_2, \ldots, t_{\mathscr{K}}]$, where $t_0 = t$, $t_{\mathscr{K}} = t_f$. The optimization objective is to minimize the deviation from the desired terminal state $\boldsymbol{\zeta}_i(t_f)$ by properly planning these $\mathscr{K}$ impulses. The proposed framework is characterized by the following three key features.

1) *Receding Horizon Implementation:* Following the receding horizon principle, only the first impulse $\Delta\boldsymbol{v}_0$ from the computed control sequence $[\Delta\boldsymbol{v}_0, \Delta\boldsymbol{v}_1, \Delta\boldsymbol{v}_2, \ldots, \Delta\boldsymbol{v}_{\mathscr{K}}]$ is implemented at time

$$\boldsymbol{\Phi}(\Delta t) = \begin{bmatrix} 4-3c & 0 & 0 & s/\omega & 2(1-c)/\omega & 0 \\ 6(s-\psi) & 1 & 0 & 2(1-c)/\omega & (4s-3\psi)/\omega & 0 \\ 0 & 0 & c & 0 & 0 & s/\omega \\ 3\omega s & 0 & 0 & c & 2s & 0 \\ 6\omega(c-1) & 0 & 0 & -2s & 4c-3 & 0 \\ 0 & 0 & -\omega s & 0 & 0 & c \end{bmatrix} \tag{6}$$

t. After execution, the control problem is resolved at the time t_1 to generate a new optimal sequence, maintaining adaptability to dynamic changes in system states and constraints.

2) *Selective Collision-Avoidance Enforcement:* To reduce computational burden, collision-avoidance constraints are enforced only over the short-term horizon $[t, t_1]$, while being relaxed over the remaining interval $[t_1, t_f]$. This selective constraint strategy balances safety assurance with real-time feasibility, ensuring efficient online computation without compromising immediate collision risk mitigation.

3) *Adaptive Impulse Maneuver Intervals:* Following an impulsive maneuver executed by deputy i at time t, the timing of subsequent maneuver, t_1, is adaptively determined by the RL agent. Leveraging the receding horizon control framework, the RL agent makes decisions based on the most recently observed system state, thereby optimizing the selection of the next maneuver time. As a result, the interval Δt between two successive impulses is not fixed, which significantly enhances the flexibility and maneuverability of the spacecraft.

We now detail the structure of the proposed learning-based MPC framework. The process begins with the definition of the deputy spacecraft's state transition model. At a given time t, the state of the deputy i is represented as $\boldsymbol{\zeta}_i(t) = [x, y, z, \dot{x}, \dot{y}, \dot{z}]^T$. The sequence of maneuver times $T = [t_0, t_1, \ldots, t_{\mathscr{K}}]$ is generated by an RL algorithm. Let $t(k) = t, t(k+1) = t_1, \ldots, t(k+\mathscr{K}) = t_{\mathscr{K}}$, where k corresponds to the discrete time index. The associated impulsive control inputs are denoted by $[\Delta\boldsymbol{v}(k), \Delta\boldsymbol{v}(k+1), \Delta\boldsymbol{v}(k+2), \ldots, \Delta\boldsymbol{v}(k+\mathscr{K})]$. The current state is expressed as $X(k) = \boldsymbol{\zeta}_i(t) + B\Delta\boldsymbol{v}(k)$. The predicted future states are denoted as $X(k+1|k), X(k+2|k), \ldots, X(k+\mathscr{K}|k)$. According to the state-space dynamics defined in (7), the future states can be computed recursively as

$$
\begin{aligned}
&X\left(k+1\,|k\right) = \boldsymbol{\Phi}\left(\Delta t_{k+1}\right) X(k) + \mathbf{B}\Delta\boldsymbol{v}\left(k+1\right)\\
&X\left(k+2\,|k\right) = \boldsymbol{\Phi}\left(\Delta t_{k+2}\right) X\left(k+1\right) + \mathbf{B}\Delta\boldsymbol{v}\left(k+2\right)\\
&\quad = \prod_{u=1}^{2} \boldsymbol{\Phi}\left(\Delta t_{k+u}\right) X(k) + \prod_{u=2}^{2} \mathbf{A}\left(\Delta t_{k+u}\right) \mathbf{B}\Delta\boldsymbol{v}\left(k+1\right)\\
&\qquad + \mathbf{B}\Delta\boldsymbol{v}\left(k+2\right)\\
&\vdots\\
&X\left(k+\mathscr{K}\,|k\right) = \prod_{u=1}^{\mathscr{K}} \boldsymbol{\Phi}\left(\Delta t_{k+u}\right) X(k)\\
&\qquad + \prod_{u=2}^{\mathscr{K}} \boldsymbol{\Phi}\left(\Delta t_{k+u}\right) \mathbf{B}\Delta\boldsymbol{v}\left(k+1\right) + \ldots\\
&\qquad + \prod_{u=\mathscr{K}}^{\mathscr{K}} \boldsymbol{\Phi}\left(\Delta t_{k+u}\right) \mathbf{B}\Delta\boldsymbol{v}\left(k+\mathscr{K}-1\right) + \mathbf{B}\Delta\boldsymbol{v}\left(k+\mathscr{K}\right)
\end{aligned}
\tag{13}
$$

where $\Delta t_{k+u} = t_{k+u} - t_{k+u-1}$, with $u \in [1, 2, \ldots, \mathscr{K}]$. To simplify notation, define the prediction and control vectors

$$
\begin{aligned}
\boldsymbol{Y} &= [X\left(k+1\,|k\right), X\left(k+2\,|k\right), \ldots, X\left(k+\mathscr{K}\,|k\right)]^T\\
\boldsymbol{U} &= [\Delta\boldsymbol{v}\left(k+1\right), \Delta\boldsymbol{v}\left(k+2\right), \ldots, \Delta\boldsymbol{v}\left(k+\mathscr{K}\right)]^T.
\end{aligned}
\tag{14}
$$

Then, the system dynamics can be expressed in compact matrix form as

$$
\boldsymbol{Y} = \mathbf{F}X(k) + \mathbf{G}\boldsymbol{U} \tag{15}
$$

where the state transition matrix $\mathbf{F}$ and the control influence matrix $\mathbf{G}$ are defined, respectively, as

$$
\mathbf{F} = \begin{bmatrix} \prod_{u=1}^{1} \boldsymbol{\Phi}\left(\Delta t_{k+u}\right) \\ \prod_{u=1}^{2} \boldsymbol{\Phi}\left(\Delta t_{k+u}\right) \\ \vdots \\ \prod_{u=1}^{\mathscr{K}} \boldsymbol{\Phi}\left(\Delta t_{k+u}\right) \end{bmatrix} \tag{16}
$$

$$
\mathbf{G} = \begin{bmatrix}
\mathbf{B} & 0 & 0 & \cdots & 0 \\
\prod_{u=2}^{2} \boldsymbol{\Phi}_{k+u}\mathbf{B} & \mathbf{B} & 0 & \cdots & 0 \\
\prod_{u=2}^{3} \boldsymbol{\Phi}_{k+u}\mathbf{B} & \prod_{u=3}^{3} \boldsymbol{\Phi}_{k+u}\mathbf{B} & \mathbf{B} & \cdots & 0 \\
\vdots & & & & \\
\prod_{u=2}^{\mathscr{K}} \boldsymbol{\Phi}_{k+u}\mathbf{B} & \prod_{u=3}^{\mathscr{K}} \boldsymbol{\Phi}_{k+u}\mathbf{B} & \prod_{u=4}^{\mathscr{K}} \boldsymbol{\Phi}_{k+u}\mathbf{B} & \cdots & \mathbf{B}
\end{bmatrix}. \tag{17}
$$

A key reason the proposed MPC framework does not require predefined reference trajectories lies in its long-term predictive horizon, which covers the entire remaining duration of the mission. Consequently, the algorithm only needs to minimize the terminal deviation from the target state $\boldsymbol{\zeta}(t_f)$, while ensuring compliance with collision-avoidance constraints. Given the target state of the deputy i at time t_f, the optimization reference is defined as

$$
\boldsymbol{R_s} = \overbrace{\left[0, 0, \ldots, \eta_1\right]^T}^{\mathscr{K}} \boldsymbol{\zeta}\left(t_f\right) \tag{18}
$$

where η_1 is a weighting parameter. The cost function for optimization is then defined as

$$
J = \left(\boldsymbol{R_s} - \boldsymbol{Y}\right)^T \left(\boldsymbol{R_s} - \boldsymbol{Y}\right) + \boldsymbol{U}^T \bar{\mathbf{R}} \boldsymbol{U} \tag{19}
$$

with $\bar{\mathbf{R}} = \eta_2 \mathbf{I}_{\mathscr{K}\times\mathscr{K}}$, where η_2 is also a weighting parameter.

Therefore, Problem 1 can be decentralized, enabling its implementation using the limited onboard computational resources available to each spacecraft in the swarm. In summary, at each time step t, each deputy spacecraft independently computes its control input by solving the following optimization problem.

Problem 2: Decentralized control problem

$$
\min_{\boldsymbol{U}} J = \left(\boldsymbol{R_s} - \boldsymbol{Y}\right)^T \left(\boldsymbol{R_s} - \boldsymbol{Y}\right) + \boldsymbol{U}^T \bar{\mathbf{R}} \boldsymbol{U} + \left\|\Delta\boldsymbol{v}(k)\right\| \tag{20}
$$

s.t.

$$
\begin{bmatrix} \mathbf{I}_{3\times3} \\ -\mathbf{I}_{3\times3} \end{bmatrix} \Delta\boldsymbol{v}\left(\tau\right) \le \begin{bmatrix} I_{\max} \\ I_{\max} \end{bmatrix}, \tau = k+1, k+2, \ldots, k+\mathscr{K} \tag{21}
$$

$$\left\|CX_i\left(k+1\,|k\right)-C\tilde{X}_j\left(k+1\,|k\right)\right\|>d_{\min}$$
$$j\neq i,\ j=1,2,\ldots,N \tag{22}$$

where $C=[\mathbf{I}_{3\times3},\mathbf{0}_{3\times3}]$ extracts the position components of the state vector, and $\tilde{X}_j(k+1|k)$ represents the predicted states of deputy j, assuming zero control input from its current state at time $t(k)=t$. Constraint (12) is incorporated into the objective function (20) as a soft constraint.

Each deputy solves its own optimization problem based solely on its observed state, enabling decentralized and parallelizable computation. This discretized control structure allows efficient utilization of onboard computational resources and enhances the autonomy of the spacecraft swarm. Moreover, given the MTS $T=[t_1,t_2,\ldots,t_{\mathscr{K}}]$ and weighting parameters η_1 and η_2, the problem becomes a convex quadratic program upon linearization of the collision constraint (22). This allows the use of standard quadratic programming (QP) solvers to obtain the optimal solution in real time. In the following section, we describe how to determine the MTS T via RL and how to leverage reachable sets to convexify the collision avoidance constraint (22).

B. Determination of the MTS T via RL

An appropriately designed MTS T not only enhances control performance but also reduces optimization complexity and mitigates the risk of convergence to singular configurations. However, deriving an effective MTS based on heuristics or predefined rules is often inefficient and unreliable. To address this challenge, we propose an RL framework capable of rapidly generating the MTS T in response to the observed system states.

The MTS selection task is formulated as a Markov decision process (MDP). The agent (denotes the deputy spacecraft) receives the observed state S as input to a neural network, which performs feature extraction and outputs an action A representing the candidate MTS. The agent then interacts with the environment by executing the action and subsequently receives a reward R. This reward is used to guide policy optimization by updating the network parameters in a direction that maximizes the expected cumulative reward. The effectiveness and training efficiency of the RL-based method hinge critically on three aspects: the representational accuracy of the observed state, the expressiveness of the neural network for feature extraction, and the relevance of the reward function to the control objective.

During the reconfiguration process, each deputy spacecraft can observe three types of information: 1) its own position and velocity, 2) the remaining mission time and its assigned target position and velocity, and 3) the relative positions of other spacecraft within the swarm. For a given deputy i, its current position and velocity at time t can be expressed as $X_i(t)=[x_i,y_i,z_i,\dot{x}_i,\dot{y}_i,\dot{z}_i]^T$. The target state is denoted by $\boldsymbol{\zeta}_i(t_f)$, and the remaining time is t_f-t. To enhance the generalizability and learning stability of the RL algorithm, all physical quantities related to length, velocity, and time are nondimensionalized through normalization. The normalization coefficients for position, velocity, and time are defined as

$$\begin{aligned}c_{\text{position}}&=\frac{1}{a_{\text{chief}}}\\ c_{\text{velocity}}&=\frac{1}{\varsigma\ m/s}\\ c_{\text{time}}&=\frac{1}{2\pi}\sqrt{\frac{\mu}{a_{\text{chief}}^3}}\end{aligned} \tag{23}$$

where a_{chief} is the semimajor axis of the chief spacecraft's orbit and $\varsigma=10$ is a tunable scaling parameter used to normalize the velocity. In the following discussion, normalized quantities are denoted with a tilde. For instance, the normalized position and velocity components are given by $\hat{x}_i=c_{\text{position}}\cdot x_i$ and $\hat{v}_x=c_{\text{velocity}}\cdot\dot{x}_i$, respectively.

The representation of relative position information poses the greatest challenge in constructing the state space. If the relative positions between a given deputy spacecraft i and all other deputies are encoded individually, then the dimensionality of the input state increases linearly with the number of spacecraft, resulting in heightened computational complexity and potential instability in the neural network. To circumvent this issue, this study employs APFs to effectively encapsulate the relative positional relationships in a compact and informative form. The APF method defines a scalar potential field over the state space, which attains a minimum at the desired configuration. The total potential field comprises two components: an attractive potential that guides each deputy toward its target, and a repulsive potential that enforces collision avoidance among the deputies. The attractive component is modeled as a quadratic function

$$\phi_{r,i}(t)=\frac{1}{2}\gamma_a\left[\boldsymbol{\rho}_i(t)-\boldsymbol{\rho}_i(t_f)\right]^T\mathbf{Q}_a\left[\boldsymbol{\rho}_i(t)-\boldsymbol{\rho}_i(t_f)\right] \tag{24}$$

where γ_a is a tunable scaling factor and $\mathbf{Q}_a$ is a positive-definite weighting matrix that shapes the potential landscape. $\boldsymbol{\rho}_i(t)$ and $\boldsymbol{\rho}_i(t_f)$ denote the current and target positions of deputy i, respectively. The repulsive potential is modeled as a Gaussian function, accounting for the proximity of other spacecraft

$$\phi_{r,i}(t)=\sum_{j\neq i,j=1}^{N}\exp\left(-\frac{\left[\boldsymbol{\rho}_i(t)-\boldsymbol{\rho}_j(t)\right]^T\mathbf{Q}_r\left[\boldsymbol{\rho}_i(t)-\boldsymbol{\rho}_j(t)\right]}{2}\right) \tag{25}$$

where $\mathbf{Q}_r$ is a shaping matrix that defines the influence range and intensity of repulsive forces. The total artificial potential $\phi_i(t)$ is the sum of the attractive and repulsive terms

$$\begin{aligned}\phi_i(t)=&\frac{1}{2}\gamma_a\left[\boldsymbol{\rho}_i(t)-\boldsymbol{\rho}_i\left(t_f\right)\right]^T\mathbf{Q}_a\left[\boldsymbol{\rho}_i(t)-\boldsymbol{\rho}_i\left(t_f\right)\right]+\\ &\sum_{j\neq i,j=1}^{N}\exp\left(-\frac{\left[\boldsymbol{\rho}_i(t)-\boldsymbol{\rho}_j(t)\right]^T\mathbf{Q}_r\left[\boldsymbol{\rho}_i(t)-\boldsymbol{\rho}_j(t)\right]}{2}\right).\end{aligned} \tag{26}$$

To encode the influence of other deputies into the state space in a concise and differentiable manner, the gradient

of the total potential field with respect to the position coordinates (x_i, y_i, z_i) is used

$$\left[\frac{\partial \phi_i(t)}{\partial x_i}, \frac{\partial \phi_i(t)}{\partial y_i}, \frac{\partial \phi_i(t)}{\partial z_i}\right]. \tag{27}$$

In addition, to capture the anticipated trend of relative motion, the partial derivatives of the potential at a future time step $t + \delta t$ under zero control input are also included

$$\left[\frac{\partial \phi_i\,(t+\delta t)}{\partial x_i}, \frac{\partial \phi_i\,(t+\delta t)}{\partial y_i}, \frac{\partial \phi_i\,(t+\delta t)}{\partial x_i}\right]. \tag{28}$$

The full-state representation $S_i(t)$, which is fed into the neural network, is defined as

$$S_i(t) = \begin{bmatrix} \frac{\partial \phi_i(t)}{\partial \boldsymbol{\rho}_i(t)} & \frac{\partial \phi_i(t+\delta t)}{\partial \boldsymbol{\rho}_i(t+\delta t)} \\ \Gamma(t) & \Gamma\,(t+\delta t) \\ \hat{x}_i & \hat{y}_i \quad \hat{z}_i \quad \hat{v}_x \quad \hat{v}_y \quad \hat{v}_z \\ & \hat{\Lambda}(t) \end{bmatrix} \tag{29}$$

where the auxiliary term $\Gamma(t)$ is defined as

$$\Gamma(t) = \left[c_{\text{position}} \cdot d_{\text{m}}(t),\ o^i_{\text{den}}(t),\ c_{\text{time}} \cdot (t_f - t)\right] \tag{30}$$

where $d_{\text{m}}(t)$ is the distance between deputy i and its closest neighbor, and $o^i_{\text{den}}(t)$ denotes the local spatial density of deputies in the vicinity of deputy i. Only spacecraft within $L = 500$ m of deputy i are included in the density $o^i_{\text{den}}(t)$, which is calculated, as shown in (31). These quantities, along with their values at $t + \delta t$, describe the evolution of the deputy's surrounding environment over the short prediction horizon

$$\begin{aligned} o^i_{\text{den}}(t) &= \sum_{j=1, j\neq i}^{N} \beta_j \frac{L - \left\|\boldsymbol{\rho}_i - \boldsymbol{\rho}_j\right\|}{0.1L} \\ \beta_j &= \begin{cases} 1 & \left\|\boldsymbol{\rho}_i - \boldsymbol{\rho}_j\right\| \leqslant L \\ 0 & \text{else.} \end{cases} \end{aligned} \tag{31}$$

Finally, the relative state vector $\Lambda(t)$ is given by

$$\Lambda(t) = X_i(t) - \zeta_i\left(t_f\right). \tag{32}$$

This term captures the deviation between the current and desired final states of deputy i. All relevant quantities in the state vector are normalized, and as a result, the complete input fed into the neural network is structured as a 4×6 matrix, compactly encoding both target-tracking objectives and obstacle-avoidance awareness. To enable each row vector in the state $S_i(t)$ to not only perceive information from other rows but also preserve its own features, we design a modified layer. It consists of a rowwise multilayer perceptron (MLP) and a lightweight cross-aware gating structure, and the data processing is as follows:

$$\begin{aligned} \boldsymbol{v}_i &= \text{ReLU}\left(\mathbf{W}_2 \cdot \text{ReLU}\left(\mathbf{W}_1 \cdot \boldsymbol{v}_i + \boldsymbol{b}_1\right) + \boldsymbol{b}_2\right) \\ \boldsymbol{g}_i &= \sigma\left(\mathbf{W}_g\left[\boldsymbol{v}_i; \bar{\boldsymbol{v}}_{\text{others}}\right]\right),\ \ \bar{\boldsymbol{v}}_{\text{others}} = \frac{1}{3}\sum_{j\neq i} \boldsymbol{v}_j \\ \boldsymbol{v}_i &= \boldsymbol{g}_i \odot \boldsymbol{v}_i + \left(1 - \boldsymbol{g}_i\right) \odot \bar{\boldsymbol{v}}_{\text{others}} \end{aligned} \tag{33}$$

where $\mathbf{W}_1$, $\mathbf{W}_2$, and $\mathbf{W}_3$ are trainable parameters and $\odot$ denotes the Hadamard product.

Given the complexity and internal dependencies among the rows of the state matrix S, employing a conventional fully connected neural network would lead to suboptimal feature extraction. To address this limitation, a simplified Transformer [51] architecture is adopted in this study for constructing the neural policy network, as illustrated in Fig. 2.

The output of the neural network, the action $A_i(t)$, is defined as

$$A_i(t) = [\Delta t_1, \Delta t_2, \ldots, \Delta t_{\mathscr{K}}] \tag{34}$$

with

$$\begin{aligned} &\sum_{k=1}^{\mathscr{K}} \Delta t_k = t_f - t \\ &t_k = t + \sum_{j=1}^{k} \Delta t_k;\ t_k \in T = [t_1, t_2, \ldots, t_{\mathscr{K}}]. \end{aligned} \tag{35}$$

To determine the optimal number of impulses, we conducted preliminary training experiments on a small-scale satellite swarm with impulse counts of 6, 8, 10, and 12. The results show that setting $\mathscr{K} = 10$ provides an effective tradeoff between achieving desirable control performance in the MPC framework and maintaining a manageable computational burden during training.

The reward function is an important component in the RL algorithm, as it guides the agent in exploring the policy space to discover the optimal control policy. In our problem, the objective of the RL is to generate an MTS that minimizes the value of (20) while avoiding collisions. To further ensure safety during the reconfiguration process, penalty terms related to the minimum distance $d_{\text{m}}(t)$ and local spacecraft density $o_{\text{den}}(t)$ are also incorporated into the reward function. The formulation of the reward function is presented in the (36) shown at the bottom of this page, where λ, ε, and φ are scaling factors, d_{m} is the minimum value of the function $d_{\text{m}}(x)$ over the interval $x \in [t, t_2]$, and the auxiliary function $F\,(d_{\text{m}})$ is a piecewise-defined function that quantifies the safety margin with respect to interagent proximity, and is given by

$$F\,(d_{\text{m}}) = \begin{cases} -\,(200 - d_{\text{m}})\,/10 - 10 & d_{\text{m}} < 100m \\ -\,(200 - d_{\text{m}})\,/10 & 100m < d_{\text{m}} < 200m \\ 0 & \text{else.} \end{cases} \tag{37}$$

$$R(t) = \begin{cases} -\lambda\left[(\boldsymbol{R}_s - \boldsymbol{Y})^T\,(\boldsymbol{R}_s - \boldsymbol{Y}) + \boldsymbol{U}^T\bar{\boldsymbol{R}}\boldsymbol{U} + \|\Delta \boldsymbol{v}(k)\|\right] + \varepsilon F\,(d_{\text{m}}) - \varphi o_{\text{den}}, & \text{if and collision free} \\ 20, & \text{if task achievement} \\ -50, & \text{else} \end{cases} \tag{36}$$

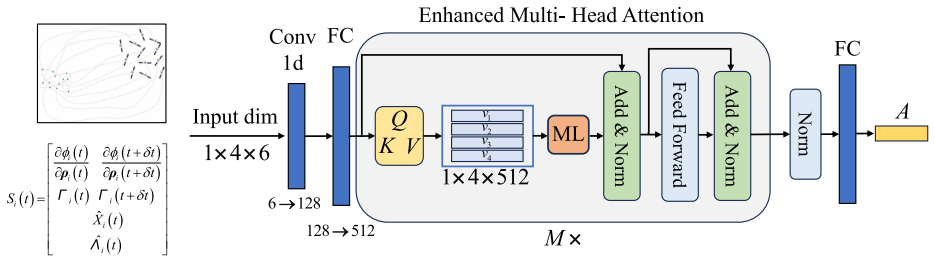

Fig. 2. Architecture of the neural network.

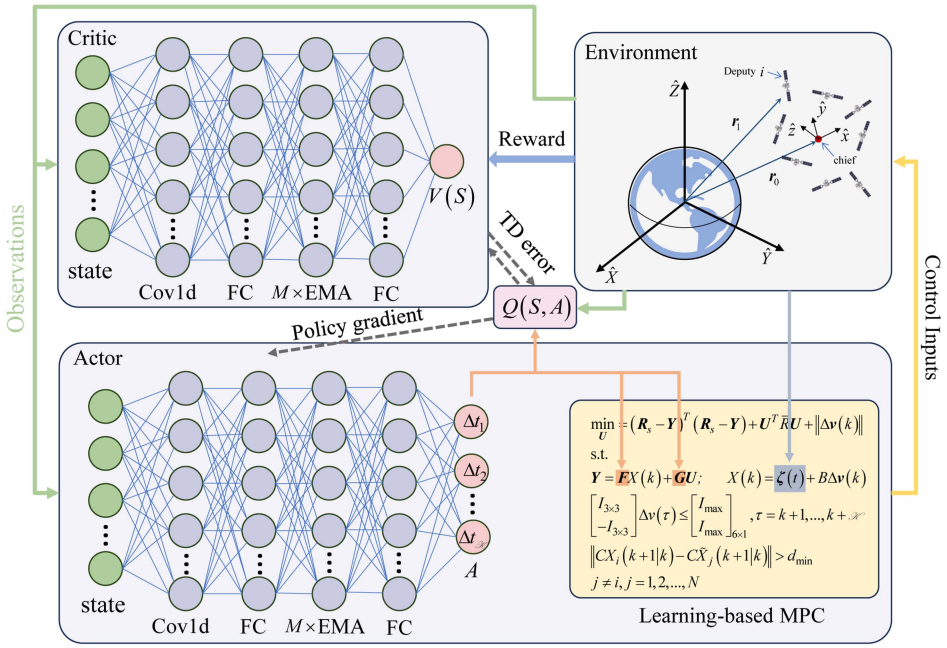

Fig. 3. Block diagram of the approach. The incorporation of the MPC method as the terminal layer within the actor network for action computation has notably increased the algorithm's robustness.

After defining all components of the MDP, the agent can interact with the environment to collect experience tuples $[S, A, R, \text{Done}, S']$, which are used to train the neural network. During training, we consider a state value function $V(S)$, a soft Q-function $Q(S, A)$, and a policy $\pi_\theta(S)$. The soft actor–critic [52] method is adopted for policy learning. The update rules are as follows:

$$J_V = \mathbb{E}_{(S,A)\sim\mathcal{D}}\left[\frac{1}{2}\big(V(S) - \big(Q(S,A) - \log\pi(A|S)\big)\big)^2\right] \tag{38}$$

where the $\mathcal{D}$ denotes the replay buffer

$$J_Q = \mathbb{E}_{(S,A)\sim\mathcal{D}}\left[\frac{1}{2}\big(Q(S,A) - \breve{Q}(S,A)\big)^2\right] \tag{39}$$

with $\breve{Q}(S,A) = R(S,A) + \gamma\mathbb{E}_{S'\sim\mathcal{D}}[V(S')]$.

The policy is optimized by minimizing the following objective:

$$J_{\pi_\theta} = \mathbb{E}_{(S,A)\sim\mathcal{D}}\left[\log\pi_\theta(A|S) - Q(S,A)\right]. \tag{40}$$

The above presents the overall framework of the proposed method. A concise flowchart, as shown in Fig. 3, is provided to illustrate the algorithm's decision making and training process. In the following section, we will discuss the convexification of collision-avoidance constraints.

C. Convexification of Collision-Avoidance Constraints

The final step in addressing the swarm reconfiguration problem is converting the collision-avoidance constraints to convex constraints. Since the employed QP solver requires affine constraints, it is necessary to first transform the original nonlinear collision-avoidance conditions accordingly. To facilitate this process, we begin by introducing the concept of the reachable set.

DEFINITION 1 (REACHABLE SET) For a system (7) with no unmodeled dynamics under the initial state $\boldsymbol{\zeta}_j(t_0)$ and the impulsive maneuver sequence $[\Delta\boldsymbol{v}_1(t_1), \Delta\boldsymbol{v}_2(t_2), \ldots, \Delta\boldsymbol{v}_k(t_k)]$, the reachable set $\aleph_j(t_0, t)$ of the deputy j is the set of all possible position states $\rho_i(t)$ at the time t

$$\aleph_j(t_0,t) = \boldsymbol{\Phi}_p(t-t_0)\boldsymbol{\zeta}_j(t_0) + \sum_{m=1}^{k}\boldsymbol{\Phi}_q(t-t_m)\Delta\boldsymbol{v}_m \tag{41}$$

where $\boldsymbol{\Phi}_p = \boldsymbol{\Phi}[0:3,:]$ and $\boldsymbol{\Phi}_q = \boldsymbol{\Phi}[3:6,3:6]$.

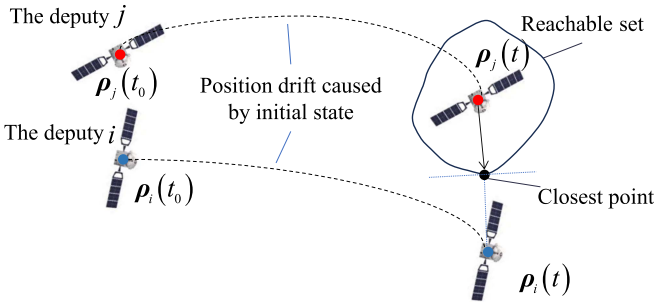

Fig. 4. 2-D shapes of reachable set. The impact of the reachable set of the deputy j on the deputy i.

However, when the number of maneuvers $k > 1$, the structure of the reachable set becomes significantly more complex and difficult to predict analytically. Fortunately, the proposed learning-based MPC framework, enhanced by RL, inherently reduces the frequency of impulsive maneuvers. This characteristic allows us to adopt a simplifying assumption to facilitate constraint convexification. Specifically, we assume that during the interval $[t_0, t]$, each deputy spacecraft j executes at most one impulsive maneuver, which occurs at the initial time t_0. Under this assumption, the reachable set of deputy j admits a tractable analytical form, thereby enabling efficient incorporation into the convex optimization problem. It is worth emphasizing that because the proposed algorithm operates in a receding horizon manner, subsequent impulses are recalculated based on the updated state after each maneuver. Thus, the single-impulse assumption in the reachable set definition does not compromise solution optimality, while substantially reducing computational complexity.

Fig. 4 schematically illustrates how deputy spacecraft i avoids a potential collision with deputy j by leveraging the concept of the reachable set. At the initial time t_0, the positions of deputies i and j are denoted as $\boldsymbol{\rho}_i(t_0)$ and $\boldsymbol{\rho}_j(t_0)$, respectively. In the absence of any control input, both spacecraft follow their natural drift trajectories, reaching positions $\boldsymbol{\rho}_i(t)$ and $\boldsymbol{\rho}_j(t)$ at a future time t. If, however, deputy j executes an impulsive maneuver at time t_0, then its potential position at time t can be approximated by a circular region representing its reachable set. To ensure safety, deputy i must maintain a minimum separation from the boundary of this region, thereby avoiding the closest possible point in the reachable set of deputy j.

Based on this concept, the procedure for convexifying the collision-avoidance constraints can be formulated as follows. At time t, deputy i observes the complete state information of all spacecraft in the swarm, denoted as $\{\boldsymbol{\zeta}_1, \boldsymbol{\zeta}_2, \ldots, \boldsymbol{\zeta}_N\}$. The RL algorithm then determines a control decision $T = [t_0, t_1, \ldots, t_{\mathscr{K}}]$ based on these observed states. According to the constraint specified in (22), deputy i must avoid collisions with all other spacecraft at the designated future time t_1.

Let the positions of all spacecraft at time t be denoted as $[\boldsymbol{\rho}_1, \boldsymbol{\rho}_2, \ldots, \boldsymbol{\rho}_i, \ldots, \boldsymbol{\rho}_N]$. At the prediction time t_1, the closest point in the reachable set of each spacecraft (excluding deputy i) to deputy i is computed, forming a set of closest points $\mathscr{R} = [\varrho_{1-i}, \varrho_{2-i}, \ldots, \varrho_{N-i}]$.

With this set $\mathscr{R}$ established, the convexification procedure begins by identifying the point $\varrho_{\mathrm{n}} \in \mathscr{R}$ that is nearest to $\boldsymbol{\rho}_i$. The vector from $\boldsymbol{\rho}_i$ to ϱn is then projected onto the x, y, and z coordinate axes. The axis along which the projection has the greatest magnitude is selected as the most critical direction of potential collision. This axis is then used to define either an upper or lower affine bound on the collision-free region for deputy i along that direction. Subsequently, all points in $\mathscr{R}$ whose projections along the selected axis are greater than or equal to that of ϱ_{n}, including ϱ_{n} itself, are removed from the set. This process is repeated iteratively to determine affine bounds in all six directions (positive and negative along the x-, y-, and z-axes), thereby constructing a convex polyhedral safe region. As a result, the original nonlinear collision-avoidance constraint $|CX_i(k+1 \mid k) - C\tilde{X}j(k+1 \mid k)| > d_{\min}$ is reformulated as a set of affine inequalities, denoted in (42). These collectively define a collision-free region $\mathscr{D}$ for deputy i within the prediction horizon. A 2-D illustration of this convexification process is provided in Fig. 5

$$\begin{aligned} x_{\min} &\le x_i\,(k+1\,|k\,) \le x_{\max} \\ y_{\min} &\le y_i\,(k+1\,|k\,) \le y_{\max} \\ z_{\min} &\le z_i\,(k+1\,|k\,) \le z_{\max}. \end{aligned} \tag{42}$$

D. Recursive Feasibility and Stability

In this section, we sequentially establish the recursive feasibility and stability of the proposed algorithm. The infeasibility of the optimization problem can arise from a single source: violation of the collision-avoidance constraints, as defined in (22). In other words, as long as the collision-avoidance constraints are satisfied at each time step, Problem 2 remains solvable throughout the execution. To ensure that this condition holds, we introduce an auxiliary constraint.

THEOREM 1 For deputy i, if Problem 2 admits a feasible solution at time step $t(k) = t$, and the auxiliary constraint given in (43) is enforced at each decision-making step of the RL algorithm, then Problem 2 remains recursively feasible at all subsequent time steps

$$\Delta t_1 \le \min\left\{t_{\lim} - t, t_f - t\right\} \tag{43}$$

where $t_{\lim}$ is the earliest time at which the reachable set of deputy i intersects with that of any other spacecraft. Δt_1 denotes the time interval to the next maneuver, as determined by the agent based on the current state.

PROOF To establish Theorem 1, we adopt a case-by-case analysis. As illustrated in Fig. 6, at time t, deputy i generates a control sequence $[\Delta \boldsymbol{v}_0^q, \Delta \boldsymbol{v}_1^q, \Delta \boldsymbol{v}_2^q, \ldots, \Delta \boldsymbol{v}_{\mathscr{K}}^q]$ based on the action vector $A^q = [\Delta t_1^q, \Delta t_2^q, \ldots, \Delta t_{\mathscr{K}}^q]$, and executes the qth impulsive maneuver $\Delta \boldsymbol{v}_0^q$ at time t. By time $t + \Delta t_1^q$, a new action $A^{q+1} = [\Delta t_1^{q+1}, \Delta t_2^{q+1}, \ldots, \Delta t_{\mathscr{K}}^{q+1}]$ is generated, it can be shown that a collision-free region $\mathscr{D}^{q+1}$ at time $t + \Delta t_1^q + \Delta t_1^{q+1}$ necessarily exists.

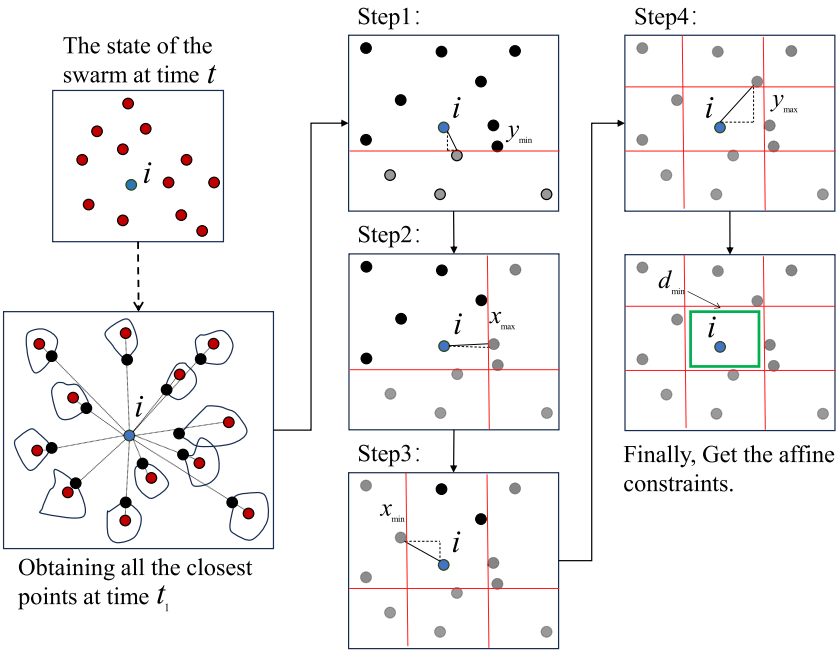

Fig. 5. 2-D example of the convexification of collision-avoidance constraints. After computing all the closest points at time t_1, the collision-avoidance constraints of the 2-D example are convexfied through the following four steps. Step 1: Identify the point closest to the deputy i, yielding the lower bound on the y-axis, $y_{\min}$. Step 2: From the remaining spacecraft, find the nearest point to the deputy i, resulting in the uupper bound on the x-axis, $x_{\max}$. Steps 3 and 4: Proceed to determine the lower bound on the x-axis, $x_{\min}$, and the upper bound on the y-axis, $y_{\max}$. Finally, these bounds form the affine constraints.

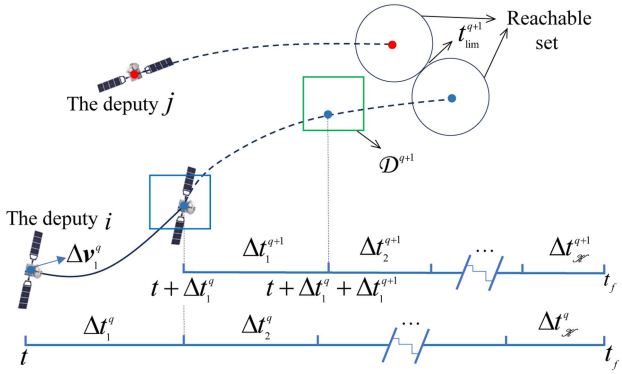

Fig. 6. 2-D shapes of reachable set. The impact of the reachable set of the deputy j on the deputy i.

Let deputy j denote the neighboring spacecraft whose reachable set first intersects with that of deputy i in the future. If the motion of deputy j does not render the collision-free region $\mathscr{D}^{q+1}$ of deputy i empty at time $t+\Delta t_1^q+\Delta t_1^{q+1}$, then the Theorem 1 holds. To verify this condition, we classify the maneuvering behavior of deputy j into the following three cases.

Case 1: Assume that deputy j executes a maneuver at time t_j, $t \le t_j < t+\Delta t_1^q$, and the next maneuver time is t_j', $t_j' > t+\Delta t_1^q+\Delta t_1^{q+1}$. The constraint (43) is satisfied for deputy j, too. Since $t_j' > t+\Delta t_1^q+\Delta t_1^{q+1}$, it follows from constraint (43) that $t_{\lim} > t+\Delta t_1^q+\Delta t_1^{q+1}$, indicating that the time at which the reachable set of deputy j intersects with that of deputy i extends beyond $t+\Delta t_1^q+\Delta t_1^{q+1}$. Therefore, the collision-free region $\mathscr{D}^{q+1}$ exists. The analysis for the scenario where deputy j performs multiple maneuvers during the interval $[t, t+\Delta t_1^q]$ follows a similar approach.

Case 2: Suppose that deputy j performs a maneuver at time t_j, where $t+\Delta t_1^q \le t_j < t+\Delta t_1^q+\Delta t_1^{q+1}$, followed by another maneuver at time $t_j' > t+\Delta t_1^q+\Delta t_1^{q+1}$. In this case, the auxiliary constraint (43) remains satisfied for deputy i. At time $t+\Delta t_1^q$, the reachable set of deputy j, as computed by deputy i, encompasses all possible positions that j may reach after performing the maneuver at t_j. Consequently, the existence of a collision-free region $\mathscr{D}^{q+1}$ is ensured. If deputy j performs multiple maneuvers during this interval, then constraint (43) still enforces persistent collision avoidance between the two spacecraft.

Case 3: Assume that deputy j performs a maneuver at time t_j with $t \le t_j < t+\Delta t_1^q$, and then executes a second maneuver at $t_j' \ge t+\Delta t_1^q$. Since the second maneuver has not yet occurred at time $t+\Delta t_1^q$, the reachable sets of deputies i and j do not yet intersect at t_j'. Under the auxiliary constraint (43), it is sufficient that $\Delta t_1^{q+1} < t_j' - t - \Delta t_1^q$ to guarantee the existence of the collision-free region $\mathscr{D}^{q+1}$. Furthermore, at time $t+\Delta t_1^q+\Delta t_1^{q+1}$, when deputy i performs its next maneuver, constraint (43) ensures that it avoids a collision with deputy j, thus confirming safety at time t_j'.

These three cases collectively encompass all possible maneuver strategies for deputy j, and Table I summarizes

Algorithm 1: Learning-Based MPC Algorithm.

At initial time t_0, let $t = t_0$, RL agent generates a MTS for each deputy, thereby constructing a maneuver time schedule $\mathscr{T}_{\text{swarm}} = [t + \Delta t^0, t + \Delta t^1, \ldots, t + \Delta t^N]$;
while $t < t_f$ **do**
  Identify the deputy i that performs the next maneuver earliest in the maneuver time schedule $\mathscr{T}_{\text{swarm}}$, i.e., $i = \text{argmin}(\mathscr{T}_{\text{swarm}})$;
  Let $t = t + \Delta t^i$, and calculating each deputy's state ;
  Generate S_i based on Eq.(29), and RL agent output an action $A_i(t)$. Generating new MTS based on $A_i(t)$ for deputy i ;
  Solve *problem 2* based on new MTS for deputy i subject to constraints (21),(42), and (43), getting the impulsive maneuver command $\Delta \boldsymbol{v}_i(t)$;
  Update the state of deputy i and the the maneuver time schedule $\mathscr{T}_{\text{swarm}}$;
  if $t_f - t < \Delta t_{stop}$ *and* $t_f - t < t_{\text{lim}}$ **then**
    The MTS of deputy i is no longer updated.
  end
end

TABLE I
Summary of Theorem 1 Under Different Maneuver Scenarios

Cases	Maneuvers	Conclusion
Case 1	A maneuver is performed during $[t, t + \Delta t_1^q]$, while no maneuver is executed during $[t + \Delta t_1^q, t + \Delta t_1^q + \Delta t_1^{q+1}]$.	Since $t'_j > t + \Delta t_1^q + \Delta t_1^{q+1}$, it follows from constraint (43) that $t_{\text{lim}} > t + \Delta t_1^q + \Delta t_1^{q+1}$, indicating that the time at which the reachable set of deputy j intersects with that of deputy i extends beyond $t + \Delta t_1^q + \Delta t_1^{q+1}$.
Case 2	No maneuver is executed during $[t, t + \Delta t_1^q]$, and one maneuver is performed during $[t + \Delta t_1^q, t + \Delta t_1^q + \Delta t_1^{q+1}]$.	At $t + \Delta t_1^q$, the reachable set of deputy j (computed by i) already encompasses all possible post-maneuver states of j. Hence, $\mathscr{D}^{q+1}$ exists. If multiple maneuvers occur, then constraint (43) still ensures persistent collision avoidance.
Case 3	Maneuvers are executed in both intervals $[t, t + \Delta t_1^q]$ and $[t + \Delta t_1^q, t_f]$.	At $t + \Delta t_1^q$, the second maneuver has not yet occurred, so reachable sets of i and j do not intersect. Under constraint (43), $\Delta t_1^{q+1} < t'_j - t - \Delta t_1^q$ suffices to guarantee $\mathscr{D}^{q+1}$. At $t + \Delta t_1^q + \Delta t_1^{q+1}$, constraint (43) further ensures safety.

the key points of Theorem 1. Moreover, by definition, the size of a spacecraft's reachable set increases monotonically with time. Consequently, the auxiliary constraint in (43) not only guarantees collision avoidance at two consecutive decision points, $t(k)$ and $t(k+1)$, but also ensures that the entire interval $[t(k), t(k+1)]$ remains collision-free. At the time t_0, the existence of a solution to Problem 2 implies that the problem remains solvable throughout the entire transfer process. At the initial stage, by imposing appropriate constraints on the initial state, a feasible solution can be easily ensured. Thus, the recursive feasibility of the proposed algorithm is established.

Given recursive feasibility, there exists a control sequence $U_c(k) = [\Delta \boldsymbol{v}(k), \Delta \boldsymbol{v}(k+1), \ldots, \Delta \boldsymbol{v}(k+M)]$ defined over the interval $[t, t_f]$ that satisfies constraints (21), (22), and (43), steering deputy i to the predicted terminal state $X_i(k+M|k)$ at time t_f. Here, the number of impulses M and the time intervals between consecutive impulses are determined by the RL agent. After sufficient training and convergence, the reward function (35) drives the value of the cost function (20) toward its minimum, ensuring that $\|X_i(k+M) - \zeta_i(t_f)\| \le \varepsilon_i$.

Based on the control sequence, we define a Lyapunov function V_k

$$V_k = \left\| X_i\left(k+M|k\right) - \boldsymbol{\zeta}_i\left(t_f\right) \right\| + \sum_{n=0}^{M} \left\| \Delta \boldsymbol{v}\left(k+n\right) \right\|. \quad (44)$$

At time $t(k+1)$, select a candidate control sequence as $U_c(k+1) = U_c(k)[1:] = [\Delta \boldsymbol{v}(k+1), \Delta \boldsymbol{v}(k+2), \ldots, \Delta \boldsymbol{v}(k+M)]$. Clearly, $U_c(k+1)$ also satisfies the constraints. Compute the Lyapunov function based on $U_c(k+1)$ at time $t(k+1)$

$$\begin{aligned} V_{k+1} &= \left\| X_i\left(k+M\,|k+1\right) - \boldsymbol{\zeta}_i\left(t_f\right) \right\| + \sum_{n=1}^{M} \left\| \Delta \boldsymbol{v}\left(k+n\right) \right\| \\ &= \left\| X_i\left(k+M\,|k\right) - \boldsymbol{\zeta}_i\left(t_f\right) \right\| + \sum_{n=0}^{M} \left\| \Delta \boldsymbol{v}\left(k+n\right) \right\| \\ &\quad - \left\| \Delta \boldsymbol{v}(k) \right\| \\ &= V_k - \left\| \Delta \boldsymbol{v}(k) \right\| \le V_k. \end{aligned} \quad (45)$$

Thus, it follows that $V_{k+1} \le V_k$, and since V_k is nonincreasing and bounded below, the system is asymptotically stable. In conclusion, the learning-based MPC algorithm is both recursively feasible and guarantees asymptotic stability. The complete algorithmic procedure is summarized in Algorithm 1.

TABLE II
Chief Spacecraft Orbit Parameters

Parameters	Values
Semi-major axis	$a \sim U\,(7000, 15000)$ km
Eccentricity	0
Inclination	$i \sim U(0, 90)^\circ$
Right ascension of the ascending node	0°
Argument of perigee	0°

IV. NUMERICAL SIMULATIONS

In this section, we present simulation results of spacecraft swarm reconfiguration using the algorithm developed in Section III. Reconfiguration scenarios involving 16 and 48 spacecraft are illustrated to demonstrate the algorithm's real-time decision-making capability, autonomous trajectory planning, and low control frequency, all achieved without reliance on a predefined reference trajectory. These results further validate the effectiveness of the proposed reachable-set-based batch processing method for handling collision-avoidance constraints. Moreover, a comparative analysis is conducted against two classical spacecraft swarm reconfiguration algorithms from the literature. The comparison highlights the advantages of the proposed method in terms of computational efficiency and fuel consumption. To assess robustness, additional simulations are performed under perturbed conditions. The results confirm that even when the policy is trained in an idealized environment, the algorithm is capable of successfully accomplishing reconfiguration tasks by continuously adapting based on real-time state observations. All simulations are implemented in Python and executed on a laptop equipped with an Intel Core i5-12400F processor (2.50 GHz) and an NVIDIA GeForce RTX 3060 GPU

$$\begin{aligned} &x = \cos(\alpha)\cos(o)L;\ y = \sin(\alpha)\cos(o)L;\ z = \sin(o)L \\ &\alpha \in [0, 2\pi],\ o \in \left[-\frac{\pi}{2}, \frac{\pi}{2}\right],\ L \sim U\,(4000, 8000)\text{m} \\ &v_i = U\,(-0.1, 0.1),\ i = x, y, z. \end{aligned} \tag{46}$$

A. Training Process of the Learning-Based MPC

The environmental parameters for the training of the RL agent are set as follows. Assume that all of the swarm spacecraft are located on Earth near-circular orbit with the orbital parameters shown in Table II. The initial position and velocity of each deputy are randomly sampled. The initialization method is given in (46). To make sure that the *problem 2* has a solution at initial time t_0, we impose constraints on the initial distance between any two spacecraft, $\|\boldsymbol{\rho}_i(t_0) - \boldsymbol{\rho}_j(t_0)\| \geqslant 200$ m. The initial velocity of each spacecraft is bounded by $\|\boldsymbol{v}_i\|_\infty < 1$ m/s. $I_{\max} = 0.5$ m/s. $t_f \sim U\,(4000, 6000)$ s, the number N of the spacecraft in the swarm is selected uniformly by $N \in \{12, 16, 20, 24\}$. The collision-avoidance distance $d_{\min} = 40$ m. $\Delta t_{\text{stop}} = 0.1 t_f$. The target formation reconfiguration of the spacecraft swarm is relative circular orbit, and the detailed computation method is provided in [22]. The radius length of the relative circular orbit is $a_r \sim U\,(300, 1500)$ m.

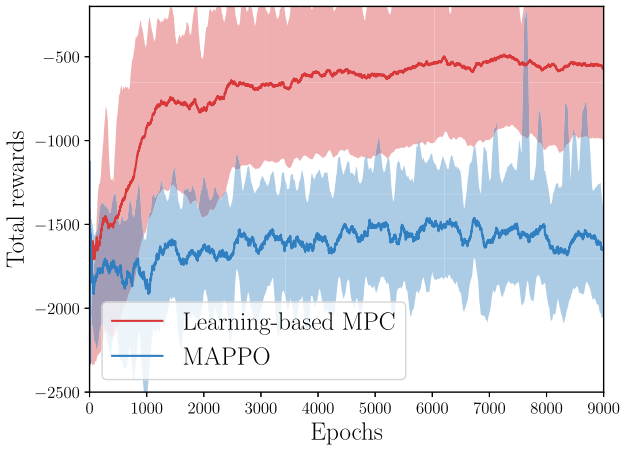

Fig. 7. Training reward curves of learning-based model predictive control (LBMPC) and MAPPO.

During training, the multi agent proximal policy optimization (MAPPO) algorithm [53] was selected as the baseline to evaluate the performance of the proposed method. The entire training process required approximately 40 h, and the results are presented in Fig. 7. As shown, our algorithm exhibits rapid convergence, reaching stability after roughly 5000 epochs. In contrast, MAPPO fails to identify a feasible solution within the same training duration and remains unconverged throughout. This efficiency stems from the following two key features of the algorithm.

1) *Full experience sharing among all agents:* Each deputy spacecraft operates as an independent agent, observing distinct states and generating different actions. Nevertheless, all agents share their experiences completely. Consequently, each iteration of the swarm generates a large amount of training data, allowing every agent to learn from the experiences of others without needing to explore all possibilities individually.

2) *Integration of RL with MPC for efficient decision-making:* The RL component is responsible for generating the MTS, while the MPC gives out the maneuver impulses. Like embedding, the MPC is the final layer of the network in the decision-making process. As a result, every action executed by the RL agent is meaningful and contributes directly to performance improvement, eliminating inefficient exploration. Therefore, the proposed algorithm demonstrates fast convergence, exhibiting high training efficiency and stability.

B. Numerical Simulations of the Learning-Based MPC for Swarm Reconfiguration

In this section, we present two numerical simulation cases to demonstrate the characteristics of the proposed algorithm. The swarm reconfigurations with 16 and 48 spacecraft are, respectively, implemented under the proposed algorithm. In both cases, we present the transfer trajectories of the spacecraft swarm and the minimum relative distances between each spacecraft and all others in the swarm. In addition, a representative deputy is selected to illustrate its minimum relative distance, the local spacecraft density, and

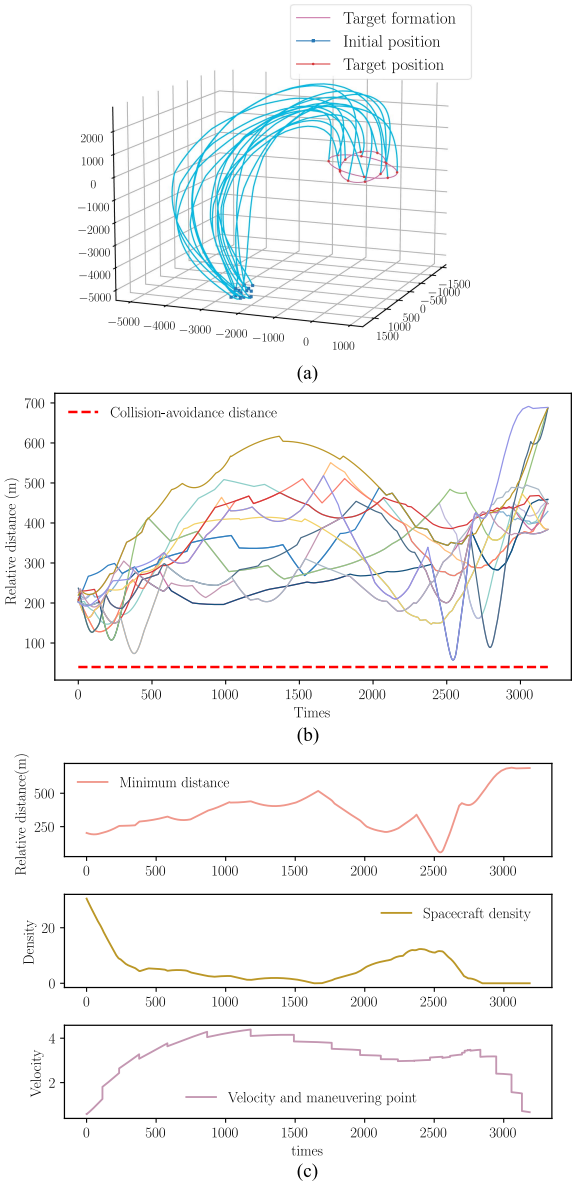

Fig. 8. Transfer process of the swarm with 16 spacecraft. (a) The swarm reconfiguration trajectories of 16 spacecraft. (b) The relative minimum distance between 16 spacecraft. (c) The transfer process of deputy 5.

the velocity profile throughout the entire transfer process, with maneuver times clearly marked on the velocity curve. All figures are recorded at 1-s intervals, demonstrating that our algorithm ensures collision-free motion not only at each maneuver time but also between consecutive maneuver epochs.

The simulation results of the swarm reconfiguration with 16 spacecraft are illustrated in Fig. 8. The target formation of the swarm consists of two relative circular orbits that are mutually perpendicular. The radii of the circular orbits are 300 and 600 m, respectively. The transfer time is $t_f = 3187$ s. The semimajor axis of the chief orbit is $a = 7000$ km. From the trajectories and the relative minimum distance curves, we can see that although the spacecraft remains in a relatively close state throughout the entire process, only a small number of spacecraft approach hazardous positions (i.e., distances close to the collision-avoidance threshold). This demonstrates the feasibility and stability of the proposed algorithm. To demonstrate the intelligence and autonomy of the algorithm, deputy 5 is selected, and all its maneuver times and corresponding impulse magnitudes are illustrated. Deputy 5 performs a total of 25 maneuvers throughout the mission, with a maximum interval of 313 s, a minimum interval of 15 s, and an average maneuver interval of 132.79 s. Compared to traditional MPC algorithms, the learning-based MPC significantly reduces the frequency of online optimization computations, thereby effectively conserving onboard computational resources. The agent determines the next maneuver time based on the real-time observed state. When the spacecraft is in a hazardous state, i.e., at risk of collision with other spacecraft in the swarm, multiple maneuvers are executed to avoid the risk, as seen around 2500 s in the third subplot of Fig. 8. In contrast, when the spacecraft is safely and steadily coasting, the maneuver intervals are long and evenly spaced, ensuring energy efficiency. The decreasing density curve indicates that deputy 5 is continuously moving toward a safer region and also reflects the transition of the swarm from a clustered state to a more dispersed distribution.

The simulation results of the swarm reconfiguration with 48 spacecraft are illustrated in Fig. 9. Unlike a swarm of 16 spacecraft, a swarm of 48 spacecraft is not encountered during the training process of the algorithm. Nevertheless, our algorithm still performs effectively, which can be attributed to the design of the state matrix S, where the relative states are described using APFs. This ensures the strong generalization capability of the algorithm. The target formation of the swarm with 48 deputies consists of four relative circular orbits that are mutually perpendicular. The radii of the circular orbits are 400, 700, 1000, and 1300 m, respectively. The transfer time is $t_f = 3190$ s. The semimajor axis of the chief orbit is $a = 15000$ km. The transfer trajectory demonstrates that even in an unfamiliar and complex environment, the proposed learning-based MPC algorithm can successfully complete the mission without guidance from a reference trajectory. This highlights the strong adaptability and high autonomy of the algorithm. Deputy 32 performs a total of 50 maneuvers during the transfer process; the maximum maneuver interval is 359 s, and the minimum interval is 3 s. Deputy 32 performed a large number of maneuvers primarily because it temporarily entered a hazardous region between 400 and 500 s. To avoid collisions during this period, it executed multiple maneuvers and ultimately succeeded in moving to a safe area, thereby completing the mission successfully. The average number of maneuvers per deputy in the swarm is 27.4, which is significantly lower than the 50 maneuvers performed by deputy 32.

C. Comparative Experiments With Existing Algorithms

In this section, we compare our algorithm with three baseline algorithms. The first algorithm (APF-based algorithm) is from [22]. It is based on APFs and is therefore

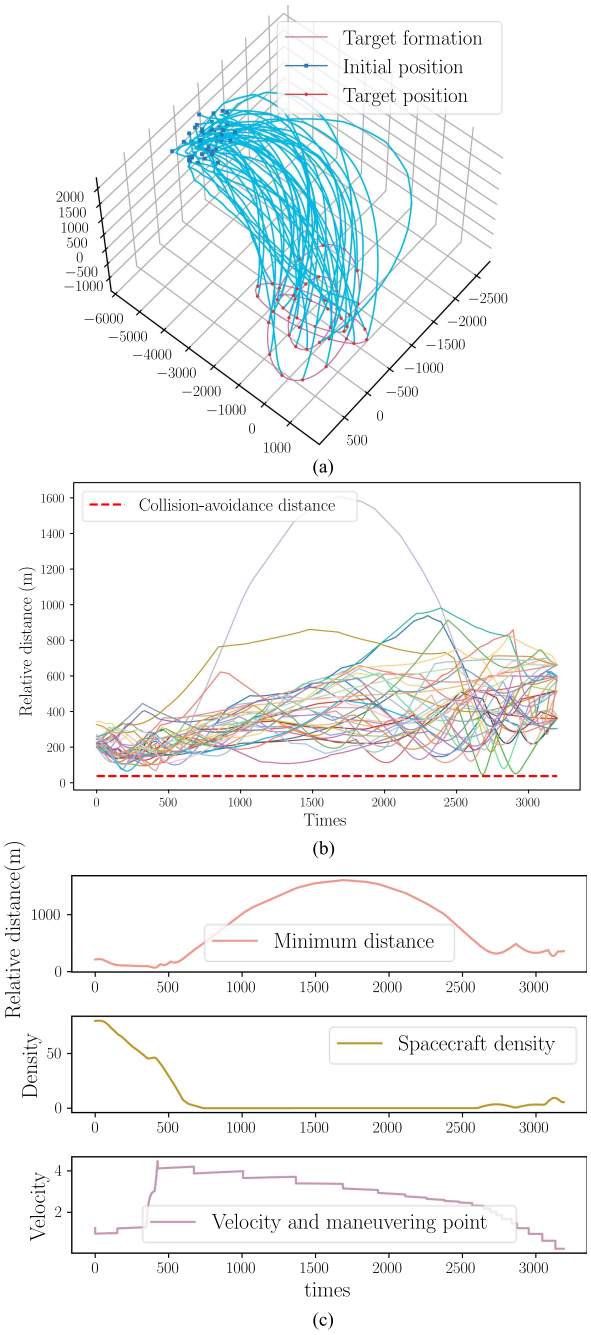

Fig. 9. Transfer process of the swarm with 48 spacecraft. (a) The swarm reconfiguration trajectories of 48 spacecraft. (b) The relative minimum distance between 48 spacecraft. (c) The transfer process of deputy 32.

capable of real-time online computation. The second algorithm (SCP-MPC algorithm) is from [29], which integrates the SCP method with MPC, enhancing computational efficiency while also demonstrating good fuel-saving performance. The third algorithm is the standard SCP numerical method commonly used for trajectory planning. These three algorithms each have their own strengths, allowing for a comprehensive comparison to highlight the characteristics of the proposed algorithm in this article. We conducted a Monte Carlo simulation campaign consisting of 500 randomized scenarios to evaluate and compare the performance

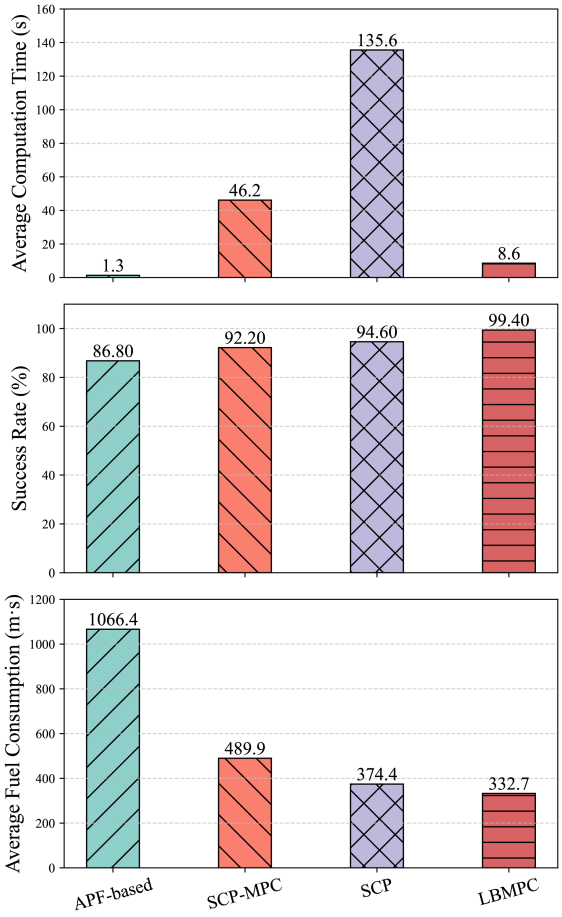

Fig. 10. Comparison results with three baseline algorithms.

of the four algorithms in terms of average computational time, success rate, and average consumption. The time interval settings for both Algorithms 1 and 3 are both $\Delta t = 30$ s. The parameters of Algorithm 2 are set according to the original paper. To reduce the computational burden of the numerical methods, the number of spacecraft in the swarm is set to 12. The comparison results are shown in Fig. 10.

Since all scenarios are randomly generated, including the transfer time t_f, some cases suffer from insufficient time, which is why none of the four algorithms achieve a 100% mission success rate. Nevertheless, our algorithm still achieves the highest success rate, as the other three baseline algorithms cannot guarantee collision-free motion between consecutive maneuver epochs. Moreover, the time interval Δt of the three baseline algorithms is fixed, making them significantly less flexible than the learning-based MPC. It can also be observed that although the APF-based algorithm has the shortest average computation time, its fuel consumption is 3.2 times higher than that of our algorithm. In addition, the fuel consumption of our algorithm is lower than that of the SCP-MPC and SCP algorithms, amounting to only 67.% and 88.9% of their respective fuel consumption.

From the perspective of practical engineering requirements, our algorithm offers greater applicability, particularly in time-critical and multiagent cooperative tasks—not

TABLE III
Success Rate Comparison of Different Methods

Methods	Success Rates		
	without δv	consider δv	drop
APF-based	86.8%	80.2%	7.60%
SCP-MPC	92.2%	88.6%	3.90%
LBMPC	99.40%	97.40%	2.01%

TABLE IV
Comparison of Terminal Convergence Errors

	APF-based	SCP-MPC	LBMPC
Position convergence errors (m)	2.02	0.80	1.58
Velocity convergence errors (m/s)	0	0	0

only in spacecraft formation reconfiguration, but also in applications involving uncrewed aerial vehicle swarms or autonomous driving systems. First, the significantly reduced computation time translates into lower computational overhead. The reduced computational time directly addresses the onboard processor's stringent real-time requirements, where available cycles are often limited due to concurrent navigation and control tasks. Second, minimized fuel consumption leads to enhanced mission efficiency and cost-effectiveness. The demonstrated reduction in fuel consumption translates into longer operational lifetime for spacecraft and increased mission flexibility, both of which are critical in resource-constrained environments. Moreover, autonomy represents a key potential advantage of the algorithm. The ability to make independent decisions enables spacecraft to perform missions without continuous ground intervention, thereby reducing reliance on human resources and operational costs.

D. Robustness Analysis

In this section, we analyze the robustness of the algorithm. Good robustness indicates a strong ability to resist disturbances, which enhances the stability of the system in practical engineering applications. During the training process of the proposed algorithm, the CW equations are used to describe the dynamical environment, and the perturbations in the environment are neglected. Therefore, in the robustness analysis, we introduce both the modeling errors ς caused by linearization and environmental perturbations ι into the dynamic model. To simplify the computation, the cumulative effect of these errors and perturbations over each 1-s interval is treated as a small impulsive maneuver $\delta \boldsymbol{v} \in \mathbb{R}^{3\times 1}$. The transition equations (5) are then updated as follows:

$$\begin{aligned} \boldsymbol{\zeta}\left(t_k\right) &= \boldsymbol{\Phi}\left(t_k - t_{k-1}\right)\boldsymbol{\zeta}\left(t_{k-1}\right) + \sum\nolimits_{i=t_{k-1}}^{t_k-1} \boldsymbol{\Phi}\left(t_k - i\right)\mathbf{B}\delta\boldsymbol{v} \\ \delta \boldsymbol{v}_{3\times 1} &= \int_t^{t+1} \boldsymbol{\varsigma}(x) + \boldsymbol{\iota}(x)dx \approx o \times N(0,1) \end{aligned} \tag{47}$$

where $o = 0.01$, $N(0, 1)$ is the standard Gaussian distribution.

Under the influence of $\delta \boldsymbol{v}$, the Monte Carlo simulation experiments in Section IV-C are repeated. The randomly generated scenarios are identical to those used in Section IV-C, and the same disturbance is applied to each algorithm during the simulation process. The mission success rates of the four algorithms are presented in Table III.

In this comparison, only the APF-based algorithm and SCP-MPC, two decentralized methods, are considered. The SCP method, as a centralized approach, becomes nearly incapable of producing reliable solutions when disturbances are introduced, making it unsuitable for direct comparison. Among the three decentralized algorithms, all possess the capability to make decisions based on observed states, which results in relatively minor decrease in mission success rates. Nevertheless, our algorithm still achieves the highest success rate. This is because our algorithm employs an adaptive and flexible maneuver interval Δt, whereas the two baseline algorithms use fixed intervals. As a result, under significant random disturbances, the baseline methods fail to adjust trajectories in a timely manner, leading to collisions. Experimental results demonstrate that even though environmental disturbances were not considered during training, the proposed algorithm maintains a high mission success rate in perturbed environments. This is attributed to the autonomous decision-making capability of the RL framework, which enables timely policy adjustments based on observed states.

We also compute the convergence error between the spacecraft's state at the final time t_f and the target state. The calculation is defined as follows:

$$e_{\text{mean}} = \frac{1}{D}\sum_{i}^{D}\left\|\boldsymbol{\chi}_i\left(t_f\right) - \boldsymbol{\chi}_{\text{target}}^{i}\right\| \tag{48}$$

where D denotes the number of Monte Carlo simulation runs and χ represents the deputy's position or velocity vector. The terminal convergence errors of the three algorithms are presented in Table IV.

From the results, 1) the velocity convergence errors are all zero, as the target configuration involves close-proximity formation flying with very small terminal velocity requirements. Consequently, spacecraft can apply relatively large impulses at t_f to fully correct any velocity errors; 2) the position convergence errors for all three algorithms are below 3 m, with SCP-MPC achieving the smallest error. The terminal position error is influenced by the time interval between the last two maneuvers. Although our algorithm currently does not outperform SCP-MPC in this regard, its performance can be improved through transfer learning. Specifically, the policy can be trained to include an additional impulse in the final phase to further reduce position errors.

V. CONCLUSION

This article proposes a reference-free, decentralized learning-based MPC algorithm for solving large-scale spacecraft swarm reconfiguration problems. The algorithm

integrates RL into MPC; the RL module predicts future MTS based on observed states, while the MPC module computes the optimal current control impulses in accordance with the predicted MTS. As a result, the algorithm enables real-time onboard decision-making with high autonomy and intelligence, eliminating the need for ground command support. By combining receding horizon control with full-horizon prediction provided by RL, the learning-based MPC can perform optimization without requiring any predefined reference trajectory. To efficiently handle collision-avoidance constraints, this article introduces a fast reachable-set-based method for constructing collision-free regions, enabling batch processing of a large number of spacecraft. Moreover, the maneuver intervals of the learning-based MPC are adaptive, allowing more flexible trajectory planning while significantly reducing computational frequency and conserving onboard computing resources. Simulation results demonstrate that the proposed algorithm completes a single reconfiguration problem in only 8.6 s, is only slightly slower than the APF-based algorithm, yet significantly faster than the other two baseline algorithms. Moreover, the fuel consumption of our algorithm is only 31.2%, 67.9%, and 88.9% of the three baseline algorithms, respectively. Even when environmental distributions and errors due to model linearization are considered, the proposed algorithm maintains a mission success rate of 97.4%. From the perspective of scalability and long-term application, the proposed algorithm holds potential for diverse engineering missions, such as autonomous health monitoring of space stations by microsatellite and nanosatellite swarms, collaborative Earth observation through small satellite constellations, and coordinated formation control for large-scale space-based solar power systems. Looking ahead, we aim to further advance this research by evaluating its robustness under realistic mission constraints, developing strategies for integration with onboard computational architectures, and exploring broader applications in next-generation autonomous space systems.

REFERENCES

[1] S.-J. Chung and F. Hadaegh, "Swarms of femtosats for synthetic aperture applications," in *Proc. 4th Int. Conf. Spacecraft Formation Flying Missions Technol.*, St-Hubert, QC, Canada, 2011.

[2] R. Chen, Y. Bai, Y. Zhao, Y. Wang, W. Yao, and X. Chen, "Closed-loop optimal control based on two-phase pseudospectral convex optimization method for swarm system," *Aerosp. Sci. Technol.*, vol. 143, 2023, Art. no. 108704.

[3] Y. Chihang, Z. Hao, and F. Weida, "Pattern control for large-scale spacecraft swarms in elliptic orbits via density fields," *Chin. J. Aeronaut.*, vol. 35, no. 3, pp. 367–379, 2022.

[4] M. Lützner et al., "Orbit design for a satellite swarm-based motion induced synthetic aperture radiometer (MISAR) in low-Earth orbit for earth observation applications," *IEEE Trans. Geosci. Remote Sens.*, vol. 60, 2022, Art. no. 1002116.

[5] F. Y. Hadaegh, S.-J. Chung, and H. M. Manohara, "On development of 100-gram-class spacecraft for swarm applications," *IEEE Syst. J.*, vol. 10, no. 2, pp. 673–684, Jun. 2016.

[6] H. Sanchez et al., "Starling1: Swarm technology demonstration," in *Proc. 32nd Annu. AIAA/USU Conf. Small Satellites*, 2018.

[7] W.-w. Cai, L.-p. Yang, Y.-w. Zhu, and Y.-w. Zhang, "Optimal satellite formation reconfiguration actuated by inter-satellite electromagnetic forces," *Acta Astronautica*, vol. 89, pp. 154–165, 2013.

[8] L. Garcia-Taberner and J. J. Masdemont, "FEFF methodology for spacecraft formations reconfiguration in the vicinity of libration points," *Acta Astronautica*, vol. 67, no. 7-8, pp. 810–817, 2010.

[9] Y. Wang, X. Chen, D. Ran, Y. Zhao, Y. Chen, and Y. Bai, "Spacecraft formation reconfiguration with multi-obstacle avoidance under navigation and control uncertainties using adaptive artificial potential function method," *Astrodynamics*, vol. 4, pp. 41–56, 2020.

[10] B. Cui, X. Chen, R. Chai, Y. Xia, and H.-S. Shin, "Trajectory planning of spacecraft swarm reconfiguration using reachable set-based collision constraints," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 60, no. 5, pp. 6474–6487, Oct. 2024.

[11] X. Bai, Y. He, and M. Xu, "Low-thrust reconfiguration strategy and optimization for formation flying using Jordan normal form," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 57, no. 5, pp. 3279–3295, Oct. 2021.

[12] F. Rossi, S. Bandyopadhyay, M. L. Mote, J.-P. de la Croix, and A. Rahmani, "Communication-aware orbit design for small spacecraft swarms around small bodies," *J. Guid., Control, Dyn.*, vol. 45, pp. 1–15, 2022.

[13] A. Koenig and S. D'Amico, "Robust and safe n-spacecraft swarming in perturbed near-circular orbits," *J. Guid., Control, Dyn.*, vol. 41, pp. 1–20, 2018.

[14] P. Pereira, B. J. Guerreiro, and P. Lourenço, "Distributed model predictive control method for spacecraft formation flying in a leader–follower formation," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 59, no. 3, pp. 3213–3223, Jun. 2023.

[15] B. Li, H. Zhang, W. Zheng, and L. Wang, "Spacecraft close-range trajectory planning via convex optimization and multi-resolution technique," *Acta Astronautica*, vol. 175, pp. 421–437, 2020.

[16] M. Kankashvar, H. Bolandi, and N. Mozayani, "Multi-agent Q-learning control of spacecraft formation flying reconfiguration trajectories," *Adv. Space Res.*, vol. 71, no. 3, pp. 1627–1643, 2023.

[17] X. Liu and K. D. Kumar, "Network-based tracking control of spacecraft formation flying with communication delays," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 48, no. 3, pp. 2302–2314, Jul. 2012.

[18] J. Chen, B. Wu, Z. Sun, and D. Wang, "Distributed safe trajectory optimization for large-scale spacecraft formation reconfiguration," *Acta Astronautica*, vol. 214, pp. 125–136, 2024.

[19] D. Morgan, S.-J. Chung, L. Blackmore, B. Acikmese, D. Bayard, and F. Y. Hadaegh, "Swarm-keeping strategies for spacecraft under J2 and atmospheric drag perturbations," *J. Guid., Control, Dyn.*, vol. 35, no. 5, pp. 1492–1506, 2012.

[20] T. Luo and M. Xu, "Orbital transfer strategy using only-accelerating maneuvers," in *Proc. Inst. Mech. Engineers, Part G: J. Aerosp. Eng.*, vol. 233, no. 6, pp. 2003–2009, 2019.

[21] C. Lippe and S. D'Amico, "Safe, delta-V-efficient spacecraft swarm reconfiguration usingLyapunov stability and artificial potentials," *J. Guid., Control, Dyn.*, vol. 45, no. 2, pp. 213–231, 2022.

[22] S. Renevey and D. A. Spencer, "Establishment and control of spacecraft formations using artificial potential functions," *Acta Astronautica*, vol. 162, pp. 314–326, 2019.

[23] Z. Wang, Y. Xu, C. Jiang, and Y. Zhang, "Self-organizing control for satellite clusters using artificial potential function in terms of relative orbital elements," *Aerosp. Sci. Technol.*, vol. 84, pp. 799–811, 2019.

[24] A. Bemporad and M. Morari, "Robust model predictive control: A survey," in *Robustness in Identification and Control*, A. Garulli and A. Tesi, Eds. London, U.K.: Springer 1999, pp. 207–226.

[25] D. Mayne, J. Rawlings, C. Rao, and P. Scokaert, "Constrained model predictive control: Stability and optimality," *Automatica*, vol. 36, no. 6, pp. 789–814, 2000.

[26] D. Celestini, D. Gammelli, T. Guffanti, S. D'Amico, E. Capello, and M. Pavone, "Transformer-based model predictive control: Trajectory optimization via sequence modeling," *IEEE Robot. Autom. Lett.*, vol. 9, no. 11, pp. 9820–9827, Nov. 2024.

[27] D. Sun, A. Jamshidnejad, and B. De Schutter, "A novel framework combining MPC and deep reinforcement learning with application to freeway traffic control," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 7, pp. 6756–6769, Jul. 2024.
[28] A. Romero, Y. Song, and D. Scaramuzza, "Actor-critic model predictive control," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2024, pp. 14777–14784.
[29] D. Morgan, S.-J. Chung, and F. Y. Hadaegh, "Model predictive control of swarms of spacecraft using sequential convex programming," *J. Guid., Control, Dyn.*, vol. 37, no. 6, pp. 1725–1740, 2014.
[30] D. Morgan, S. Chung, and F. Hadaegh, "Decentralized model predictive control of swarms of spacecraft using sequential convex programming," *Adv. Astronautical Sci.*, vol. 148, pp. 3835–3854, 2013.
[31] Z. Sun, B. Wu, D. Wang, and J. Chen, "Event-triggered model predictive control of spacecraft formation," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 7696–7711, 2025.
[32] H. Ren, H. Gui, and R. Zhong, "Efficient fuel-optimal multi-impulse orbital transfer via contrastive pre-trained reinforcement learning," *Adv. Space Res.*, vol. 75, no. 10, pp. 7377–7396, 2025.
[33] Q. Qu, K. Liu, W. Wang, and J. Lü, "Spacecraft proximity maneuvering and rendezvous with collision avoidance based on reinforcement learning," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 58, no. 6, pp. 5823–5834, Dec. 2022.
[34] Z. Yang et al., "Model-based reinforcement learning and neural-network-based policy compression for spacecraft rendezvous on resource-constrained embedded systems," *IEEE Trans. Ind. Informat.*, vol. 19, no. 1, pp. 1107–1116, Jan. 2023.
[35] L. Xu, G. Zhang, S. Qiu, and X. Cao, "Reinforcement learning-based multi-impulse rendezvous approach for satellite constellation reconfiguration," *Acta Astronautica*, vol. 224, pp. 325–337, 2024.
[36] J. Sun, Y. Meng, J. Huang, F. Liu, and S. Li, "Distributed cooperative control with collision avoidance for spacecraft swarm reconfiguration via reinforcement learning," *Acta Astronautica*, vol. 205, pp. 95–109, 2023.
[37] A. Sabol, K. Yun, M. Adil, C. Choi, and R. Madani, "Machine learning based relative orbit transfer for swarm spacecraft motion planning," in *Proc. IEEE Aerosp. Conf.*, 2022, pp. 1–11.
[38] Y. Meng, C. Liu, J. Zhao, and J. Huang, "Obstacle-avoidance distributed reinforcement learning optimal control for spacecraft cluster flight," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 61, no. 1, pp. 443–456, Feb. 2025.
[39] L. Hewing, K. P. Wabersich, M. Menner, and M. N. Zeilinger, "Learning-based model predictive control: Toward safe learning in control," *Annu. Rev. Control Robot., Auton. Syst.*, vol. 3, pp. 269–296, 2020.
[40] L. Brunke et al., "Safe learning in robotics: From learning-based control to safe reinforcement learning," *Annu. Rev. Control Robot., Auton. Syst.*, vol. 5, pp. 411–444, 2022.
[41] A. Aswani, H. Gonzalez, S. S. Sastry, and C. Tomlin, "Provably safe and robust learning-based model predictive control," *Automatica*, vol. 49, no. 5, pp. 1216–1226, 2013.
[42] M. Tanaskovic, L. Fagiano, and V. Gligorovski, "Adaptive model predictive control for linear time varying MIMO systems," *Automatica*, vol. 105, pp. 237–245, 2019.
[43] E. Terzi, L. Fagiano, M. Farina, and R. Scattolini, "Learning-based predictive control for linear systems: A unitary approach," *Automatica*, vol. 108, 2019, Art. no. 108473.
[44] M. Neumann-Brosig, A. Marco, D. Schwarzmann, and S. Trimpe, "Data-efficient autotuning with Bayesian optimization: An industrial control study," *IEEE Trans. Control Syst. Technol.*, vol. 28, no. 3, pp. 730–740, May 2020.
[45] S. Gros and M. Zanon, "Data-driven economic NMPC using reinforcement learning," *IEEE Trans. Autom. Control*, vol. 65, no. 2, pp. 636–648, Feb. 2020.
[46] U. Rosolia and F. Borrelli, "Learning model predictive control for iterative tasks a data-driven control framework," *IEEE Trans. Autom. Control*, vol. 63, no. 7, pp. 1883–1896, Jul. 2018.
[47] F. Berkenkamp, M. Turchetta, A. P. Schoellig, and A. Krause, "Safe model-based reinforcement learning with stability guarantees," in *Proc. Int. Conf. Neural Inf. Process. Syst.*, 2017, pp. 908–919.
[48] L. Zhang, R. Zhang, T. Wu, R. Weng, M. Han, and Y. Zhao, "Safe reinforcement learning with stability guarantee for motion planning of autonomous vehicles," *IEEE Trans. Neural Netw. Learn. Syst.*, vol. 32, no. 12, pp. 5435–5444, Dec. 2021.
[49] J. F. Fisac, A. K. Akametalu, M. N. Zeilinger, S. Kaynama, J. Gillula, and C. J. Tomlin, "A general safety framework for learning-based control in uncertain robotic systems," *IEEE Trans. Autom. Control*, vol. 64, no. 7, pp. 2737–2752, Jul. 2019.
[50] J. Wang, Z. Chen, Y. Zhao, Y. Bai, and X. Chen, "Fast approximation of spacecraft relative motion reachable set based on modal decomposition," *J. Guid., Control, Dyn.*, vol. 47, no. 12, pp. 2621–2630, 2024.
[51] A. Vaswani et al., "Attention is all you need," in *Proc. Neural Inf. Process. Syst.*, 2017, pp. 6000–6010.
[52] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, "Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1861–1870.
[53] C. Yu et al., "The surprising effectiveness of PPO in cooperative multi-agent games," in *Proc. 36th Int. Conf. Neural Inf. Process. Syst.*, 2022, pp. 24611–24624.

He Ren received the B.E. degree in spacecraft design from the School of Mechanics and Aerospace Engineering, Dalian University of Technology, Dalian, China, in 2020. He is currently working toward the Ph.D. degree in space craft design with the School of Astronautics, Beihang University, Beijing, China.

His research interests include multiagent reinforcement learning, learning-based model predictive control, and spacecraft swarm formation reconfiguration.

Rui Zhong received the Ph.D. degree in spacecraft design from the Beijing University of Aeronautics and Astronautics, Beijing, China, in 2011.

From 2011 to 2013, he was a Postdoctoral Fellow with the Department of Earth and Space Science and Engineering, York University, Toronto, ON, Canada. Since 2013, he has been an Assistant Professor with the School of Astronautics, Beijing University of Aeronautics and Astronautics, where he is currently an Associate Professor. He has authored more than 80 articles. His research interests include dynamics and control of space tether systems, spacecraft control based on intelligent algorithms, and space human–machine collaboration technology.

Dr. Zhong is a Member of the Chinese Association of Automation, and an Editorial Board Member for IJSPACESE.

Haichao Gui received the B.S. and Ph.D. degrees in aerospace engineering from Beihang University, Beijing, China, in 2009 and 2014, respectively.

He was a Crew Member and the Payload Specialist of Shenzhou-16 manned spaceflight mission, and spent 154 days in space. He is currently a Professor with the School of Astronautics, Beihang University. He is also an Astronaut. His research interests include spacecraft dynamics, nonlinear estimation and control, and space robotics.

Dr. Gui was the recipient of the honorable title of "Hero Astronaut," the "Third-Class Space Merit Medal," and the 18th China Youth Science and Technology Medal.